PROJECT REPORT

통합 보안 관제 시스템 구축 프로젝트

결과 보고서

ALEPH 3조

EngineSquare

강아름 · 김라연 · 유지원(팀장)

1. 프로젝트 개요

본 프로젝트는 가상의 B2B API 플랫폼 기업 EngineSquare 를 대상으로, 기업 API·IoT 환경의 로그 수집부터 보안 이벤트 탐지와 자동 대응까지 연결하는 통합 보안 관제 시스템을 구축하는 프로젝트이다.

1-1. 추진 배경

기업의 API 서비스와 IoT 장치가 연결된 환경에서는 포트스캔, 무차별 대입, 유출 API 키 재사용 및 비정상 장비 제어 등 다양한 위협이 발생할 수 있다. 시스템별 로그가 분산되면 공격의 진행 과정을 파악하기 어렵고, 대량 로그가 집중될 경우 처리 지연이나 유실로 탐지에 필요한 정보가 누락될 수 있다.

이에 로그 전달의 안정성과 탐지 이후의 대응을 함께 고려하여, 공격 재현·로그 수집·분석·자동 대응·인시던트 확인을 하나의 흐름으로 연결하도록 구성하였다.

1-2. 프로젝트 목적

Middleware 를 통해 보안 로그를 안정적으로 수집·버퍼링·전달하고, Graylog 와 Wazuh 를 활용해 이상 사용과 공격 흐름을 분석하는 것을 목적으로 한다. 탐지 결과는 n8n 과 연계하여 키 폐기·교체, IP 차단 및 IoT 보호 등의 대응으로 연결한다.

공용 웹앱에서는 인시던트와 최근 이벤트를 통합 조회하고, 관리자 화면에서 API 키와 차단 상태를 관리할 수 있도록 하였다. 공격 요청의 응답, 수집 로그, 자동 대응 실행 이력과 화면 표시를 대조하여 구현 결과를 검증한다.

1-3. 서비스 환경 및 구현 범위

EngineSquare 는 고객사에 API 를 제공하고 데이터센터의 물리 보안 IoT 장치가 연계된 가상의 기업 환경이다. API 키와 인증 정보, 관리 기능, 로그 관제 인프라 및 IoT 제어 기능을 주요 보호 대상으로 설정하였다.

구분	핵심 구현 범위
시나리오 1	로그 버퍼링 Middleware: 대량 로그 수집·완충·전달과 전송 안정성 검증
시나리오 2	유출 API 키 탐지: 키·출발지 IP 검증과 키 교체·IP 차단 자동 대응
시나리오 3	킬체인·IoT: 단계별 공격 로그 상관분석과 네트워크·장비 대응 연계
공용 환경	메인·로그인·게시판·대시보드·관리자 화면과 공용 API·MySQL 연동

<표 1-1> 핵심 구현 범위

1-4. 기대 효과

안정적인 로그 전달로 분석에 필요한 정보를 확보하고, 개별 이벤트와 단계별 공격 흐름을 함께 확인할 수 있다. 또한 탐지 결과와 자동 대응·관리 화면을 연결함으로써 반복적인 대응 작업을 줄이고, 운영자가 사건과 처리 상태를 한곳에서 확인할 수 있도록 한다.

2. 보호 자산 및 3 대 위협 시나리오

EngineSquare 의 API 서비스와 IoT 환경을 기준으로 보호 자산을 선정하였다. 로그 처리 인프라의 가용성, API 접근의 정당성 및 단계별 공격의 연계 여부를 세 시나리오로 나누어 다룬다.

2-1. 주요 보호 자산

보호 자산	보호 필요성
API 키·인증 정보	유출된 키가 서비스 접근에 악용되지 않도록 키 상태와 사용 출발지를 검증해야 한다.
API 데이터·서비스	비인가 요청으로부터 데이터를 보호하고 정상 사용자의 API 이용을 유지해야 한다.
로그 수집·분석 인프라	대량 이벤트와 전송 장애 상황에서도 탐지에 필요한 로그를 안정적으로 보관·전달해야 한다.
관리 기능·사건 기록	키·차단 상태와 인시던트 정보를 일관되게 관리하고 운영자가 처리 결과를 확인할 수 있어야 한다.
IoT 장치·제어 기능	비정상 제어가 장비와 물리 환경에 영향을 주지 않도록 공격 흐름을 분석하고 대응을 연계해야 한다.

<표 2-1> 주요 보호 자산

2-2. 시나리오별 위협과 대응 방향

시나리오	위협 상황	대응 방향
시나리오 1 Middleware	스캔·무차별 대입 등으로 로그가 집중되어 처리 지연과 전송 실패가 발생하는 상황	수집·버퍼링·전송 속도 제어와 재시도로 로그 전달의 안정성을 확보한다.
시나리오 2 유출 API 키	정상 발급된 키가 허용되지 않은 IP 에서 재사용되는 상황	키·출발지 IP 를 검증해 요청을 거부하고 키 폐기·교체와 공격 IP 차단을 연계한다.
시나리오 3 킬체인·IoT	동일 공격자의 정찰·유출 키 재사용·IoT 제어가 연속 발생하는 상황	단계별 로그를 상관분석하고 조건 충족 시 CRITICAL 판정과 네트워크·장비 대응을 연계한다.

<표 2-2> 시나리오별 위협과 대응 방향

2-3. 시나리오 간 연계

시나리오 1 은 보안 이벤트를 수집·전달하는 기반을 제공하고, 시나리오 2 는 API 키 이상 사용을 탐지하여 개별 대응을 수행한다. 시나리오 3 은 API 공격을 포함한 단계별 로그를 동일 공격자 기준으로 분석한다. 공용 대시보드는 사건과 대응 결과를 통합 표시하며, 세부 구현과 검증 결과는 각 시나리오 절에서 설명한다.

3. 시스템 구성

세 시나리오는 공용 환경(웹앱·API·DB)과 시나리오별 전용 구성요소를 함께 사용한다. 아래는 역할별 구성요소와 사용 기술을 정리한 것이며, 각 구성요소 간 데이터 흐름은 5 장(통합 아키텍처)에서 다룬다.

3-1. 구성요소 및 역할

계층	구성요소	역할 / 기술
프론트엔드	공용 웹앱	메인·로그인·게시판·대시보드·관리자 화면
백엔드	Flask 공용 API + MySQL	API 키 검증, 인시던트·티켓·차단 상태 저장 및 조회
로그 수집	팀 Middleware (시나리오 1)	비동기 HTTP 수집, SQLite WAL 브로커, 우선순위 드레인, 다중 Graylog 전송
로그 분석	Graylog (시나리오 2·3 전용 Event Definition)	API 키 오남용 탐지, 킬체인 상관분석 (CRITICAL/WARNING)
로그 분석(보조)	Wazuh	원본 JSON 로그 실시간 수집·규칙 기반 탐지 (100310~100312)
자동 대응	n8n (시나리오 2·3 전용 워크플로우)	Webhook 수신, 분기 처리, 외부 시스템 호출 (API/SSH/HTTP)
네트워크 보안	pfSense	공격 IP WAN 차단(SSH easyrule, 시나리오 3)
IoT	ESP32-C3 SuperMini	MicroPython HTTP 서버, 상태 LED·부저로 대응 상태 가시화
공격 재현	Kali, Postman, test_killchain.py 등	시나리오별 공격 트래픽·이벤트 재현

<표 3-1> 시스템 구성요소 및 역할

3-2. 시나리오 전용 구성요소

공용 구성요소(웹앱·API·DB) 외에, 시나리오마다 다음과 같은 전용 구성요소를 추가로 사용한다.

시나리오	전용 구성요소
시나리오 1	비동기 수집기, SQLite WAL 브로커, 스로틀링 컨슈머 워커, 서킷 브레이커 클라이언트
시나리오 2	Flask /api/data 보호 API, Wazuh 사용자 정의 규칙(100310~100312), n8n /automation/revoke-key 연동
시나리오 3	Graylog Killchain CRITICAL/WARNING Event Definition, n8n 킬체인 워크플로우, pfSense, ESP32-C3

<표 3-2> 시나리오 전용 구성요소

4. 시나리오별 구현

공용 환경(로그인·게시판·대시보드·관리자 화면, 공용 API·MySQL)을 기반으로, 세 시나리오를 각각 구현한 뒤 공용 인시던트 체계로 통합하였다. 이하 4-1~4-3 에서 시나리오별 구현을 다룬다.

4-1. 시나리오 1 — 인프라(로그 버퍼링)

대규모 공격 트래픽 수집 영속성 확보 및 하위 관제 시스템(SIEM/SOAR) 부하 평탄화를 목표로 한다.

4-1-1. 위협 시나리오 및 실측 검증

대량 공격 트래픽 유입 시 SIEM 과부하 및 서비스 마비 위험

포트 스캔이나 무차별 대입 공격으로 대량 보안 이벤트가 유입될 때 로그를 상위 Graylog 와 OpenSearch 로 직접 전송하면 Netty 수신 창구와 OpenSearch 색인 스레드 풀이 순차적으로 포화된다. 이 과정에서 패킷이 폐기되거나 연결이 강제 단절된다. 허니토큰 접근이나 킬체인 로그가 누락되어 Graylog 상관분석과 n8n SOAR 자동 대응이 실패한다.

동시에 관제 지연이 메인 API 서버로 번져 서비스 장애를 유발한다. API 서버가 보안 로그를 동기 HTTP 방식으로 전송하므로, SIEM 응답이 지연되면 워커 스레드가 고갈되어 정상 비즈니스 API 요청까지 응답 불능 상태에 빠진다.

상용 Graylog GELF HTTP 의 배치 파싱 제약 규명

상용 Graylog 6.1 의 공식 GELF HTTP 인풋은 단일 JSON 메시지 수신을 기본 규격으로 삼는다. 클라이언트가 개행 구분 JSON(NDJSON)으로 100 건을 묶어 전송할 경우, Netty Jackson 스트림 파서는 첫 번째 JSON 객체 1 건만 역직렬화하여 색인하고 나머지 99 건은 에러 없이 폐기한다. 반면 HTTP 계층에는 정상 수신을 뜻하는 202 Accepted 를 반환한다.

실제 gl-graylog 컨테이너와 OpenSearch 클러스터에 100 건 NDJSON 배치를 200 회(총 20,000 건) 전송하여 실측한 결과, 200 회의 HTTP 202 응답을 수신했으나 OpenSearch 에 실제 색인된 문서는 200 건에 불과했다. 19,800 건(99.0%)이 소실되었다. 따라서 HTTP 응답 코드에 배치 크기를 곱해 성공 건수를 산출하던 기존 벤치마크는 허위 0% 유실을 보고하는 결함이 있었다. 본 검증 체계에서는 추정 계산을 배제하고, OpenSearch 고속 Scroll API(size=10000)와 Python set()을 통해 전수 실측 기준을 확립했다.

다운스트림 장애 시 단건 카오스 벤치마크 실측 비교

1 Event = 1 HTTP POST 단건 전송, 동시 연결 세마포어, 3.0 초 SLA 타임아웃, 오픈루프 도착률 스케줄링 조건에서 다운스트림 장애(docker pause)를 주입하고 양측 파이프라인의 내구성을 실측 비교했다. OpenSearch Scroll API 전수 덤프를 통해 단 1 건의 오차도 없이 실제 적재량과 고유 건수를 집계했다.

시험 규모	측정 지표	직접 전송 (Direct Graylog)	미들웨어 적용 (SOAR Middleware)
시나리오 A (500 건 / 3 초 장애, 20 동시 워커)	클라이언트 전송 완료 시간	3.89 초 (128 req/s)	2.51 초 (199 logs/s, WAL)
시나리오 A (500 건 / 3 초 장애, 20 동시 워커)	순수 서비스 지연 (평균 / P99)	234.75 ms / 3,032.1 ms	9.89 ms / 27.5 ms

시나리오 A (500 건 / 3 초 장애, 20 동시 워커)	체감 지연 시간 (대기 포함 평균 / P99)	2,162.90 ms / 3,022.6 ms	9.86 ms / 39.8 ms
시나리오 A (500 건 / 3 초 장애, 20 동시 워커)	클라이언트 수락 / 실패 건수	295 건 수락 / 205 건 실패 (41.0%)	500 건 전량 수락 / 0 건 실패 (100%)
시나리오 A (500 건 / 3 초 장애, 20 동시 워커)	OpenSearch DB 최종 적재 건수	312 건 적재 (312 고유)	500 건 적재 (500 고유)
시나리오 A (500 건 / 3 초 장애, 20 동시 워커)	엔드투엔드 최종 유실률	37.6% (188 건 영구 유실)	0.0% (0 건 유실, 무손실 보존)
시나리오 A (500 건 / 3 초 장애, 20 동시 워커)	중복 적재율 및 최종 판정	0.0% (UNPROTECTED, 188 lost)	0.0% (0 LOST, 0 DUPS Observed)
시나리오 B (20,000 건 / 15 초 장애, 50 동시 워커)	클라이언트 전송 완료 시간	117.66 초 (170 req/s)	117.67 초 (170 logs/s, WAL)
시나리오 B (20,000 건 / 15 초 장애, 50 동시 워커)	순수 서비스 지연 (평균 / P99)	60.42 ms / 1,049.2 ms	12.01 ms / 60.9 ms
시나리오 B (20,000 건 / 15 초 장애, 50 동시 워커)	체감 지연 시간 (대기 포함 평균 / P99)	461.33 ms / 3,014.2 ms	12.24 ms / 68.2 ms
시나리오 B (20,000 건 / 15 초 장애, 50 동시 워커)	클라이언트 수락 / 실패 건수	17,757 건 수락 / 2,243 건 실패 (11.2%)	20,000 건 전량 수락 / 0 건 실패 (100%)
시나리오 B (20,000 건 / 15 초 장애, 50 동시 워커)	OpenSearch DB 최종 적재 건수	18,356 건 적재 (18,356 고유)	20,052 건 적재 (20,000 고유 보존)
시나리오 B (20,000 건 / 15 초 장애, 50 동시 워커)	엔드투엔드 최종 유실률	8.2% (1,644 건 영구 유실)	0.0% (0 건 유실, 무손실 보존)

시나리오 B (20,000 건 / 15 초 장애, 50 동시 워커)	중복 적재율 및 최종 판정	0.0% (UNPROTECTED, 1,644 lost)	+0.3% (+52 건 At-Least-Once 정상 재전송)

<표 4-1-1> 다운스트림 장애 시 직접 전송 vs 미들웨어 실측 비교

다운스트림이 정지하면 Direct 전송 방식에서는 커넥션 풀 슬롯 50 개가 고갈된다. 새로 유입되는 요청이 3.0 초 SLA 내에 슬롯을 얻지 못하고 대기열에서 탈락한다. 실측 결과 단기 장애(시나리오 A, 3 초)에서 188 건(37.6%), 장기 장애(시나리오 B, 15 초)에서 1,644 건(8.2%)이 영구 유실되었다.

반면 미들웨어를 거치면 클라이언트는 하위 상태와 무관하게 로컬 Ingestor WAL 에 평균 12.01ms 만에 기록하고 즉시 반환된다. 체감 지연 시간 역시 평균 12.24ms(P99 68.2ms)로 안정적인 수치를 유지했다. 20,000 건 전량을 유실 없이 수용하여 고유 데이터 20,000 건을 온전히 보존했다. 10 초를 넘는 장기 다운타임(15 초) 환경에서는 컨슈머 워커의 정상 타임아웃 재전송으로 인한 중복 52 건(+0.3%)이 관측되어 At-Least-Once 복원력 동작을 실측 확인했다.

4-1-2. 구현 방식

상위 서비스 응답 지연을 방지하고 Graylog 부하를 제어하기 위해 시스템을 수집기, 경량 브로커, 컨슈머 워커, 공유 메트릭 스토어 및 서킷 브레이커로 역할을 분리했다. 기동 시 환경변수와 운영 설정의 유효성을 사전 검증한다(PipelineSettings).

비동기 HTTP 수집기 및 보안 계층

수집기는 요청 본문의 필수 필드 유효성을 검증하고 타임스탬프를 초 단위 Unix 시간으로 정규화한다. 미정의 필드는 원본 페이로드 하위로 흡수해 보존한다(SecurityLogEvent). 인증 계층은 정적 API 키의 상수 시간 비교(hmac.compare_digest) 방식과 HMAC-SHA256 전자서명 방식을 동시에 지원한다. 전자서명 방식은 본문과 타임스탬프 무결성을 검증하며, 300 초 슬라이딩 윈도우와 메모리 캐시(ReplayProtectionCache)를 통해 동일 서명의 재사용 공격을 방어한다.

인증이 완료된 요청은 API 키 해시값을 키로 삼아 토큰 버킷 속도 제한을 적용한다. 브로커 적재 전에는 페이로드를 재귀 탐색해 비밀번호나 토큰 등 민감 정보를 "[REDACTED]"로 마스킹한다. SOAR 자동 차단이 참조할 IP 식별자는 최상위 필드로 분리해 보존한다. 침해 경보 지연을 막기 위해 허니토큰 접근과 위급 심각도 로그는 우선순위 토픽으로, 일반 로그는 표준 토픽으로 즉시 분기한다.

SQLite WAL 기반 경량 브로커

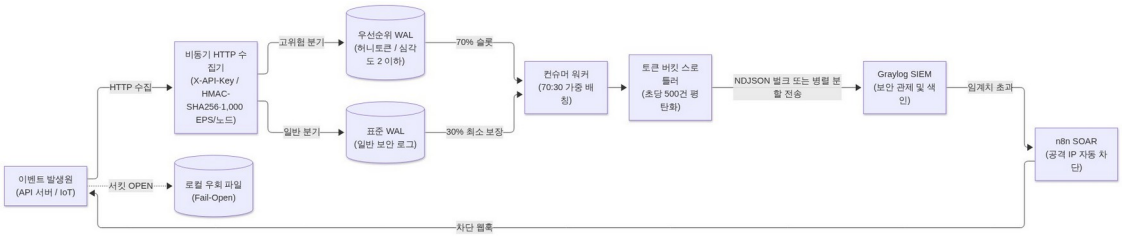
단일 호스트에서 외부 인프라 의존성 없이 디스크 영속성을 확보한다. 출발지 IP 의 CRC32 해시값으로 파티션을 고정 배정하여 동일 공격자의 킬체인 로그 순서를 보장한다. 수집 API 는 인메모리 큐에 메시지를 넣고 즉시 HTTP 202 응답을 반환한다. 백그라운드 스레드가 배치를 묶어 단일 트랜잭션으로 커밋하고 연속 오프셋을 부여한다(SQLiteBrokerStorage).

배치 커밋 락 경합 시 수집 API 의 이벤트 루프가 블로킹되지 않도록 파티션 수와 적체량은 백그라운드 태스크가 갱신한 캐시값만 참조한다. 데이터베이스 설정에는 PRAGMA busy_timeout = 15000 과 cache_size = -64000(64MB)을 적용해 락 대기 및 I/O 성능을 최적화했다. 큐 용량을 초과하면 즉시 HTTP 503 을 반환하여 상위 서비스에 백프레셔를 가한다.

스로틀링 컨슈머 워커 및 전송 제어

컨슈머 워커는 표준 큐의 최소 배출 비율을 보장하면서 우선순위 큐를 먼저 배정(70:30)하여 고위험 경보 지연을 줄이고 일반 로그 기아를 방지한다(BrokerBatchPoller). 배치 내 메시지 수만큼 토큰을 차감해 Graylog 유입량을 초당 500 건으로 제어한다. 원본 발생 시각과 고유 식별자(_event_id)를 주입해 GELF 규격으로 변환한다. 상용 Graylog 6.1 의 단건 파싱 제약에 맞춰 20 개 스레드 기반 병렬 단건 전송 모드를 기본 운용한다.

분할 전송 시에는 pending_indices 로 개별 메시지 결과를 추적한다. 성공(202)한 건은 제외하고 실패한 항목만 선별 재시도하여 하위 중복 부하를 최소화한다. 전송 실패 시에는 세션 유지를 위해 파티션을 일시 정지한 상태로 지수 백오프를 수행한다. Graylog 수신 성공이나 DLQ 격리 완료가 확인된 후에만 오프셋을 동기 커밋한다. 상위 클라이언트는 관제 장애 시 서킷을 열어 로컬 우회 파일에 기록함으로써 본래 비즈니스 서비스를 격리한다(SafeSecurityLogClient).



<그림 4-1-1> 파이프라인 전체 아키텍처 및 데이터 흐름도

4-1-3. 주요 설정값

컴포넌트	설정 항목	설정값	설계 역할 및 목적
수집기	인증 노드(API 키)별 수집 속도	초당 1,000 건 (버스트 2,000 건)	단일 노드 자원 독점 방지 및 다중 키 분산 시 4,448+ EPS 수집
수집기	보안 인증 방식	이중 인증 (API Key / HMAC-SHA256)	타이밍 공격 방지, 전송 무결성 보장 및 재전송 공격 차단
수집기	우선순위 분기 조건	Syslog 심각도 2 이하 / 허니토큰	긴급 침해 경보의 목표 전달 지연 시간(2.0 초 이내) 보장
브로커	인메모리 버퍼 큐	최대 10 만 건	디스크 쓰기 병목 시 수집 API 의 지연 없는 즉시 반환 지원
브로커	디스크 배치 쓰기	최대 250 건 또는 2ms 주기	초당 5,000 건 버스트 수집 흡수 및 SQLite WAL 디스크 I/O 최적화
브로커	DB 락 대기 / 캐시 크기	15 초 대기 (busy_timeout) / 64MB 캐시	트랜잭션 경합 시 이벤트 루프 락 방지 및 읽기 성능 최적화
컨슈머	마이크로 배치 폴링	최대 100 건 또는 100ms 대기	네트워크 왕복 횟수 감소 및 배치 처리 효율화
컨슈머	가중 드레인 비율	우선순위 70% : 표준 최소 30%	고위험 경보 우선 방출 및 일반 로그 기아 현상 방지

컨슈머	토큰 버킷 방출 속도	초당 500 건 (메시지 수 비례 차감)	하위 Graylog 수신 대기열 포화 방지 및 트래픽 평탄화
컨슈머	전송 지수 백오프	1 초 ~ 최대 30 초 (최대 8 회)	하위 서버 장애 시 네트워크 부하 완화 및 파티션 일시 정지
컨슈머	전송 모드	병렬 분할 20 스레드 (GELF_SPLIT_POST=True)	상용 Graylog 6.1 단건 파싱 제약 준수 및 실패 선별 재시도
클라이언트	인메모리 큐 / 서킷 브레이커	최대 1,000 건 / 3 회 연속 실패 시 OPEN	Fail-Open 보장 및 관제 장애의 비즈니스 서비스 전파 차단

<표 4-1-2> 주요 설정값 요약

4-1-4. 아키텍처 트레이드오프 및 상용 확장 로드맵

임베디드 SQLite WAL 채택 배경과 단일 쓰기 직렬화 한계

상용 분산 브로커인 Kafka 나 RabbitMQ 는 수 GB 의 메모리와 별도 데몬 관리가 필요하여 단일 VM 이나 메모리 제한(384MB) 환경에 부적합하다. 본 시스템은 외부 의존성이 없는 단일 파일 기반 SQLite WAL 을 채택하여 최소한의 자원으로 디스크 영속 버퍼링과 파티셔닝 스트리밍 기능을 구현했다.

단일 Uvicorn 프로세스(--workers 1) 환경에서는 인메모리 버퍼 큐, 백그라운드 단일 커밋 스레드, 15 초 대기(busy_timeout), 64MB 캐시 설정을 통해 내부 스레드 간 락 경합을 방지했다. 동시 연결 50 개 환경에서 20,000 건 전량을 유실 없이 수용함을 실측 확인했다.

다만 SQLite WAL 은 다중 읽기를 지원하지만 쓰기 트랜잭션(BEGIN IMMEDIATE)은 전역 파일 수준에서 한 번에 1 개 프로세스만 진입할 수 있다. 멀티 프로세스 워커(--workers 4) 구성 시 독립된 OS 프로세스 간의 파일 락 경합으로 sqlite3.OperationalError: database is locked 가 발생한다. NFS 같은 네트워크 파일시스템 환경에서는 공유 메모리 색인(.shm) 손상 위험이 존재하므로 다중 호스트 수평 확장에 제약이 따른다.

초당 500 건 평탄화에 따른 일반 로그 지연 및 교차 레인 순서 역전

Graylog 부하 방지를 위한 초당 500 건 전송 상한과 70:30 가중 드레인 정책으로 인해 폭주 구간에서 표준 토픽 대기열의 체류 시간이 증가한다. 이로 인해 긴급 경보와 일반 로그 간 네트워크 도착 순서가 역전될 수 있다.

이를 해결하기 위해 GELF 변환 시 원본 이벤트 발생 시각(timestamp)을 보존했다. 수신단인 Graylog 에서 네트워크 도착 시각이 아닌 실제 발생 시각 기준으로 윈도우 상관분석을 수행하므로 순서 역전으로 인한 오탐을 방지한다.

최소 1 회 전송 보장과 상용 Graylog 멱등성 한계

본 미들웨어 파이프라인의 전송 보장 수준은 At-Least-Once 이다. 다운스트림 장애나 네트워크 타임아웃 발생 시 오프셋 커밋 전에 재전송을 시도하므로 하위 시스템에 동일 로그가 중복 전달될 수 있다.

시나리오 A 의 3 초 다운타임 환경에서는 컨슈머 워커의 HTTP 읽기 타임아웃(10 초)이 장애 시간보다 길어 연결을 성급히 끊지 않고 대기했다. 파티션 일시 정지를 통해 재전송 폭주를 억제하여 중복 0 건(0.0%)을 기록했다.

반면 시나리오 B 의 15 초 장기 다운타임 환경에서는 10 초 타임아웃을 초과하여 소켓이 단절되면서 미수신 확인 배치를 재전송했다. 이에 따라 OpenSearch 실측 결과 20,052 건 중 고유 데이터 20,000 건 전량을 보존하면서 재전송에 따른 중복 52 건(+0.3%)이 관측되었다.

상용 Graylog 6.1 GELF HTTP 인풋은 클라이언트 메시지 ID 기반 중복 제거를 지원하지 않고 매 수신마다 내부 UUID 를 발급한다. 따라서 엔드투엔드 멱등성을 확보하려면 OpenSearch 색인 파이프라인에서 _id: ctx._event_id 매핑을 적용해야 한다.

상용 프로덕션 전환 로드맵

도메인 로직과 브로커 계층을 분리하여 Kafka 표준 클라이언트(confluent-kafka)의 핵심 인터페이스(Producer, Consumer, AdminClient)와 호환되는 덕 타이핑 구조로 구현했다. 수집 라우터, 민감정보 마스킹, 토큰 버킷 같은 비즈니스 로직을 수정하지 않고 스토리지 백엔드만 교체할 수 있다.

처리량 요구치가 초당 10,000 건 수준인 중간 규모 환경에서는 redis-py 기반 Redis Streams 백엔드로 전환하여 파일 락 병목을 해소하고 수만 EPS 처리량을 확보한다. 초당 100,000 건 이상의 대규모 분산 환경에서는 confluent-kafka 드라이버를 활성화하여 Apache Kafka 또는 Redpanda 클러스터로 전환한다. 이를 통해 파티션 수평 확장과 멱등성 수집 파이프라인을 구축한다.

4-2. 시나리오 2 — 인증·인가(유출 API 키)

4-2-1. 구현 목적

정상 발급된 API 키가 외부로 유출되어 허용되지 않은 IP 에서 사용되는 상황을 재현하였다. API 키의 유효성과 요청 출발지 IP 를 함께 검증하고, 이상 사용이 탐지되면 키 폐기·재발급과 공격 IP 차단까지 자동으로 수행하도록 구현하였다.

보호 대상은 EngineSquare 공용 API 와 API 가 제공하는 데이터이다. 공격자가 유효한 키를 확보하더라도 허용되지 않은 IP 에서는 접근을 거부하고, 탐지 이후 추가 사용을 제한하는 것을 목표로 하였다.

4-2-2. 구성 및 처리 흐름

Flask API 서버, 팀 Middleware, Graylog, n8n, MySQL 및 공용 웹앱을 연동하였다. Wazuh 는 원본 JSON 로그를 별도로 수집·분석하는 경로로 구성하였다. 자동 대응은 Graylog 탐지와 n8n 호출을 통해 수행하며, Wazuh 는 별도 규칙 검증 경로로 현재 구성에서 n8n 을 직접 호출하지 않는다.

4-2-3. 주요 구현 내용

API 키 인증 및 출발지 IP 검증

보호 API /api/data 는 X-API-Key 헤더와 요청 출발지 IP 를 검사한다. 활성 키를 허용 IP 에서 사용하면 HTTP 200 을 반환한다. 정상 키라도 비허용 IP 에서 사용하면 HTTP 403 과 "Client IP is not allowed" 응답을 반환한다. 인증용 원본 키와 관리용 식별자를 분리하여, 원본 키는 DB 에 해시로 저장하고 로그에는 api_key_id 를 사용해 인증값이 보안 이벤트에 직접 노출되지 않도록 하였다.

보안 로그 수집 및 이상 사용 탐지

비허용 IP 사용 시 api_key_compromised 이벤트를 생성한다. 발생 시각, 키 식별자, 출발지 IP, 요청 경로, HTTP 상태 코드 및 탐지 사유를 포함한 로그를 팀 Middleware 를 통해 Graylog 로 전달한다. Graylog 는 해당 이벤트와 HTTP 403 을 기준으로 탐지하여 시나리오 2 전용 n8n Webhook 을 호출한다.

키 폐기·재발급 및 공격 IP 차단

n8n 은 API 서버의 /automation/revoke-key 를 호출한다. 공용 API 는 기존 키 폐기, 교체 키 발급 및 공격 IP 차단을 처리한다. 키와 차단 상태는 MySQL 에서 조회하므로 서버 재시작 없이 다음 요청부터 반영된다. 교체 키 원본은 관리자에게 전달되기 전까지 암호화하여 임시 보관하고, 관리자가 새 키를 확인하면 원본 전달용 암호문을 삭제한다.

인시던트 및 공용 화면 연동

인시던트는 INC-API 형식으로 관리한다. 동일 이벤트 유형·출발지 IP·키 식별자의 요청은 기존 티켓에 연결하고 횟수를 누적한다. 대시보드에는 정상·거부 이벤트와 대응 조치를 표시하고, 관리자 화면에서는 키 상태·교체 키·차단 IP 를 확인할 수 있다. 자동 대응 완료와 티켓 종료는 구분되며, 티켓 종료가 키 복원이나 IP 차단 해제를 의미하지는 않는다.

Wazuh 규칙 적용

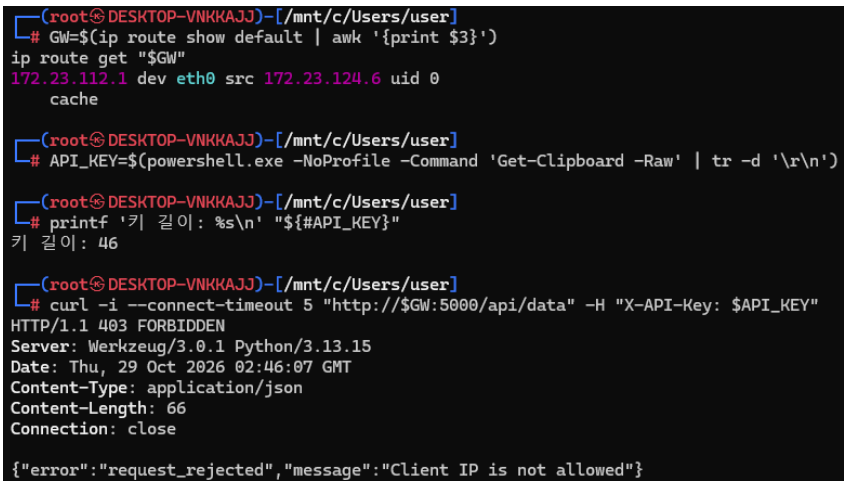
Wazuh JSON 디코더와 기본 부모 규칙을 활용하여 사용자 정의 규칙을 추가하였다.

규칙 ID	확인 대상	레벨
100310	비허용 IP 의 API 키 사용 및 HTTP 403	10
100311	정상 API 요청 허용 및 HTTP 200	3
100312	자동 대응 완료 및 HTTP 200	3

<표 4-2-1> Wazuh 사용자 정의 규칙

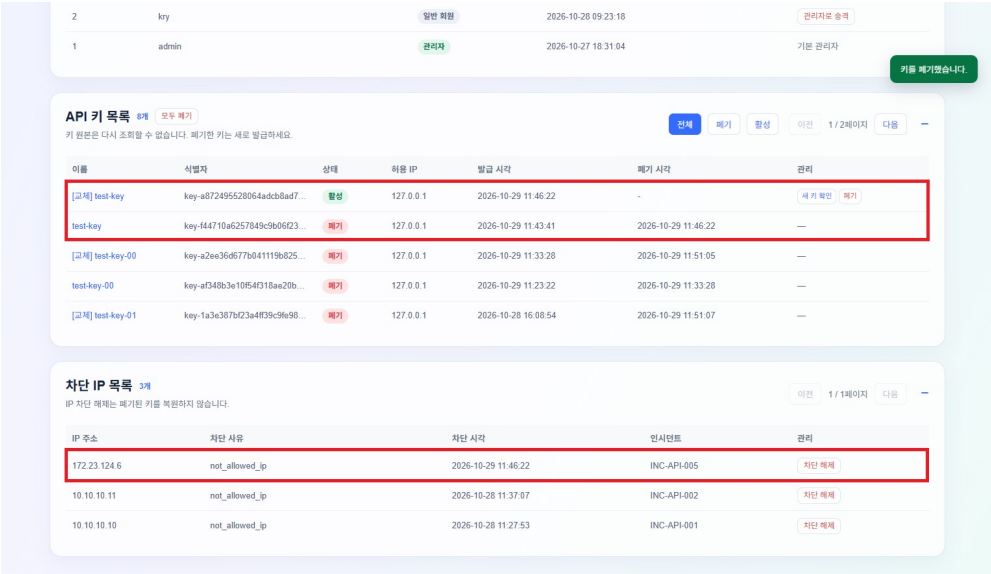
4-2-4. 재현 및 검증 결과

허용 IP 를 127.0.0.1 로 지정한 실습용 키를 발급하고 Windows Postman 에서 정상 요청을 보냈다. 이어서 같은 키를 WSL Kali 에서 사용하였다 — Kali 요청에는 IP 재현 헤더를 넣지 않고 실제 출발지 IP 172.23.124.6 을 적용하였다.



<그림 4-2-1> Kali 실제 출발지 IP 에서 유출 키 사용을 재현한 HTTP 403 응답

폐기 키의 HTTP 401 검증은 허용 IP 에서 수행하였다 — 차단된 Kali IP 에서는 IP 차단 검사가 먼저 적용되어 폐기 키를 사용하더라도 HTTP 403 이 반환될 수 있기 때문이다. n8n 실행 내역에서 키 폐기·재발급과 IP 차단, 결과 확인 및 대시보드·알림 전달 단계의 성공을 확인하였다.



<그림 4-2-2> 기존 키 폐기·교체 키 활성 상태와 INC-API-005 에 연결된 IP 차단 기록

증적에서 정상 요청, 비허용 IP 접근 및 자동 대응 완료 로그를 확인하였으며, 공격 이벤트는 INC-API-005 티켓과 연결하였다. 관리자 화면에서는 기존 키 폐기와 교체 키 발급, 공격 IP 차단을 함께 확인하였다

Wazuh 실시간 탐지 검증

Kali 의 비허용 API 키 사용 요청을 Wazuh 가 실시간으로 수집·탐지한 것을 확인하였다. 규칙 100310(레벨 10)이 적용되었으며, 로그에서 공격 IP, HTTP 403, 탐지 사유 not_allowed_ip 및 인시던트 INC-API-011 을 확인하였다. 이 결과는 과거 로그 재처리가 아닌 신규 요청의 실시간 탐지 결과이며, 기존 INC-API-005 증적과는 별도의 추가 검증이다.

검증 단계	요청 및 확인 내용	확인 결과
정상 접근	허용 IP 에서 활성 키 사용	200 / authorized
비허용 IP	Kali 에서 동일 키 사용	403 / IP 접근 거부
로그 탐지	Middleware·Graylog 에서 이벤트 확인	IP·키 식별자·시각 확인
자동 대응	n8n 실행 및 관리자 상태 확인	키 폐기·교체·IP 차단
폐기 키 재사용	허용 IP 에서 기존 키 사용	401 / API key is revoked
이용 재개	허용 IP 에서 교체 키 사용	200 / authorized

<표 4-2-2> 시나리오 2 검증 단계별 결과

4-2-5. 구현 결과 및 한계

유효한 API 키라도 출발지 IP 에 따라 접근을 제한하고, 이상 사용 탐지 이후 키 교체와 IP 차단까지 이어지는 자동 대응 흐름을 검증하였다. 또한 결과를 공용 DB 와 웹 화면에 반영하여 운영자가 사건과 대응 상태를 확인할 수 있도록 하였다. 다만 이번 검증은 비허용 IP 의 키 사용을 중심으로 수행하였으며, 허용된 환경에서 발생하는 키 악용까지 판단하려면 요청량·접근 자원·사용 패턴 등을 종합하는 기준이 추가로 필요하다.

4-3. 시나리오 3 — 상관분석·하드웨어(킬체인)

구분	내용
핵심 트리거	Graylog Event Definition "Killchain CRITICAL" — client_ip 별 그룹핑, event_type 3 종 이상, 10 분 윈도우, 1 분 주기
오케스트레이터	n8n 워크플로우 "시나리오 3 - 킬체인 상관분석 대응" — Webhook 수신 후 CRITICAL 분기로 팬아웃
대응 액션	pfSense WAN 차단(SSH easyrule) · ESP32 Lockdown · 공용 대시보드 티켓 생성 · Discord 봇 알림
하드웨어	ESP32-C3 SuperMini + LED 4 색(파랑/초록/노랑/빨강) + 능동 부저, MicroPython HTTP 서버로 동작
인증	ESP32 POST 엔드포인트는 X-Auth-Token 헤더 필요, pfSense 는 SSH 키 (Ed25519) 인증
재현 스크립트	test_killchain.py critical — recon_scan → api_key_compromised → device_control 자동 전송 (시연에서는 spike_test_demo.py 로 정찰·인증 공격 이벤트를, test_killchain.py device 로 장치 제어 이벤트를 전송)

<표 1> 시나리오 3 핵심 요약

4-3-1. 개요

시나리오 3 는 시나리오 1~2 에서 개별적으로 탐지된 위협 이벤트가 동일 공격자(client_ip)로부터 일정 시간 내 연쇄적으로 발생했을 때, 이를 Graylog 상관분석 규칙으로 자동 탐지하고 n8n 을 통해 물리적 하드웨어(ESP32-C3 기반 상태 표시 장치)가 실시간으로 대응하도록 구현한 파트이다. 단일 로그

하나만으로는 위협으로 보기 어려운 이벤트들도, 같은 IP 에서 서로 다른 유형의 이벤트가 짧은 시간 내 3 종 이상 모였다는 패턴은 정찰 → 침투 → 장치 제어로 이어지는 Kill Chain 의 진행으로 해석할 수 있다는 점에 착안하였다.

본 시나리오의 기획 의도는 보안 관제 결과를 가장 직관적인 형태로 보여주는 데 있다. 보안 회사에는 공격이 지속적으로 유입되며, 온라인 대시보드와 메시지 알림이 이미 널리 쓰이고 있으나, 벽에 걸린 시계처럼 보안 상태를 LED 색으로 한눈에 보여주는 방식이 가장 즉각적이라고 판단하였다. 또한 공격은 온라인 서비스에만 국한되지 않고 출입문과 같은 물리 장치에 대한 API 공격으로도 이어질 수 있으므로, 이를 즉시 감지하여 물리적으로 표현하는 IoT 시스템을 구현하고자 하였다.

본 파트의 목표는 다음 세 가지로 정리된다. 첫째, Graylog Event Definition 을 이용한 Kill Chain 상관분석 규칙을 설계한다. 둘째, 탐지 결과를 n8n 웹훅으로 전달하여 티켓 생성·알림·장치 제어·차단까지 이어지는 자동 대응 오케스트레이션을 구현한다. 셋째, 대응 결과를 사람이 눈으로 즉시 확인할 수 있도록 실물 LED·부저 하드웨어로 가시화한다. 이 과정에서 소프트웨어 상관분석 로직과 물리적 하드웨어 반응을 하나의 파이프라인으로 연결하는 것이 다른 시나리오와 구분되는 시나리오 3 만의 과제였다.

4-3-2. 위협 시나리오 및 대응 단계 정의

대응 단계는 탐지된 공격 진행도에 따라 4 단계로 정의하였다. 단계가 올라갈수록 LED 색상이 달라지도록 설계하여, 시스템 상태를 소프트웨어 로그를 열어보지 않고도 즉시 식별할 수 있도록 하였다.

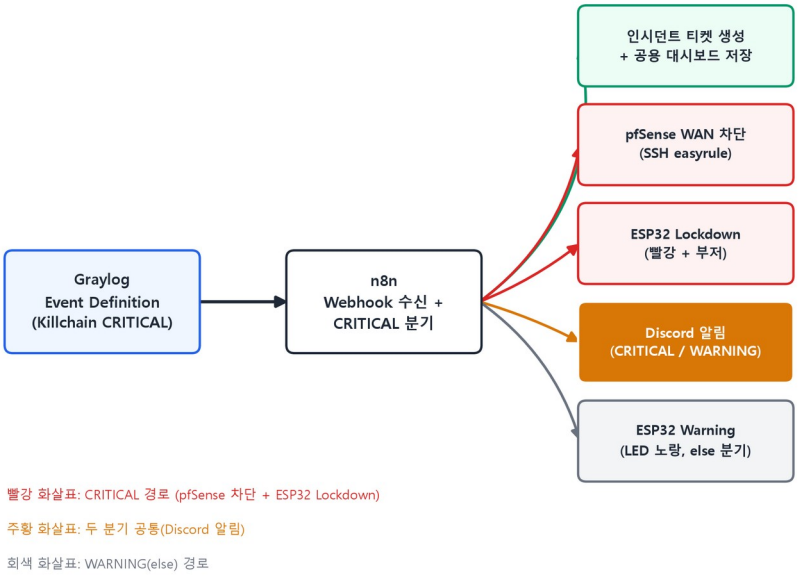
단계	의미	상태 LED
정상	공격 징후 없음	초록
경고(1~2 단계 탐지)	정찰(recon_scan/PORT_SCAN) 또는 인증 정보 탈취 (BRUTE_FORCE/api_key_compromised) 등 일부 단계만 포착	노랑
CRITICAL(3 단계 이상 확정)	서로 다른 유형의 공격 이벤트가 동일 IP 에서 3 종 이상 10 분 이내 발생 — 장치 제어(device_control)까지 포함된 완전한 킬체인으로 판단	빨강 + 부저
수동 해제	조장 승인 후 정상 복귀	초록

<표 2> 위협 단계와 대응 LED 상태

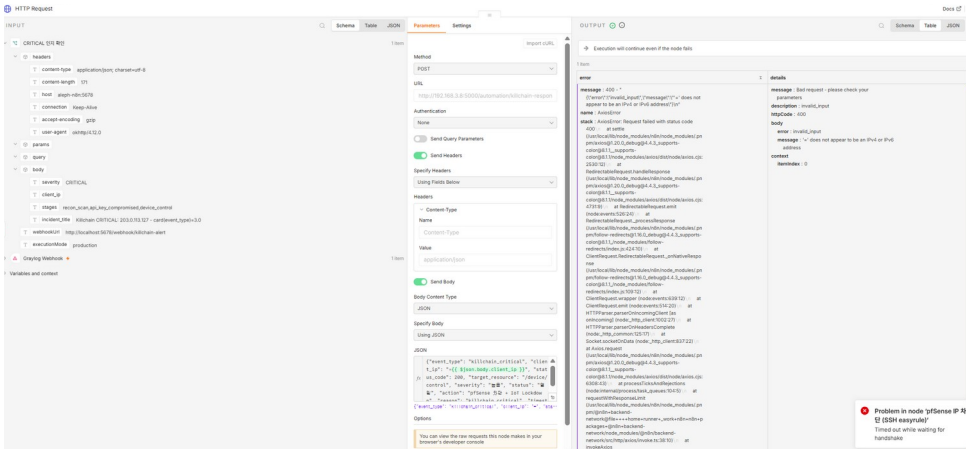
경고 단계와 CRITICAL 단계를 분리한 이유는, 정찰이나 자격 증명 탈취 시도 자체는 아직 실제 피해로 이어지지 않은 전조 신호이기 때문이다. 이를 CRITICAL 과 동일하게 취급하면 과도한 알림(alert fatigue)으로 실제 침해 상황의 긴급성이 희석될 위험이 있어, 단계별 대응 강도를 달리하였다.

4-3-3. 시스템 아키텍처

n8n 은 Graylog 탐지 알림을 받는 즉시 중앙 오케스트레이터 역할을 수행하며, ESP32 하드웨어 제어·공용 대시보드 인시던트 기록·pfSense 차단·Discord 봇 알림을 병렬로 팬아웃하는 구조로 설계하였다. Graylog 가 각 시스템을 직접 제어하는 것이 아니라, n8n 이 단일 진입점에서 모든 대응 액션을 조율한다는 점이 이 아키텍처의 핵심이다.



<그림 1> 시나리오 3 전체 대응 아키텍처



<그림 2> n8n 워크플로우 실제 화면 — Graylog Webhook → CRITICAL 인지 확인 분기

이러한 팬아웃 구조를 선택한 이유는, 탐지 즉시 수행해야 하는 대응이 한 가지가 아니라 기록(티켓)·차단(pfSense)·가시화(LED)·공지(Discord 봇)로 성격이 서로 다르기 때문이다. n8n 한 곳에서 분기와 병렬 호출을 모두 담당하게 함으로써, 각 개별 시스템(ESP32, 대시보드, pfSense, Discord 봇)은 서로의 존재를 알 필요 없이 독립적으로 동작할 수 있도록 결합도를 낮추었다.

4-3-4. 하드웨어 구성 — ESP32-C3 SuperMini

상태를 사람이 즉시 식별할 수 있도록 LED 4 색과 능동 부저로 구성된 독립형 IoT 장치를 제작하였다. 장치는 MicroPython 기반으로 Wi-Fi 에 연결된 뒤 HTTP 서버로 동작하며, 별도의 PC 연결 없이 USB 전원만 공급되면 자동으로 부팅 시퀀스를 실행하고 n8n 의 요청을 직접 수신한다.

준비물

부품	수량	비고
ESP32-C3 SuperMini	1	MicroPython 펌웨어 설치 완료 상태
LED 5mm (파랑·초록·노랑·빨강)	각 1개	긴 다리 = +(anode), 짧은 다리 = −(cathode)
220Ω 저항	4 개	색띠 무관, 전부 같은 값
액티브 부저 모듈 (SMG-A 계열, 3핀)	1	핀 순서 GND / VCC / S — 모듈 뒷면 각인 확인
브레드보드 (830point)	1	중앙 홈 기준 좌우로 보드 실장
점퍼선 (수-수)	최소 2 개	부저 S 용 1개, GND 용 1개
USB-C 데이터 케이블	1	충전 전용 케이블 아닌 것

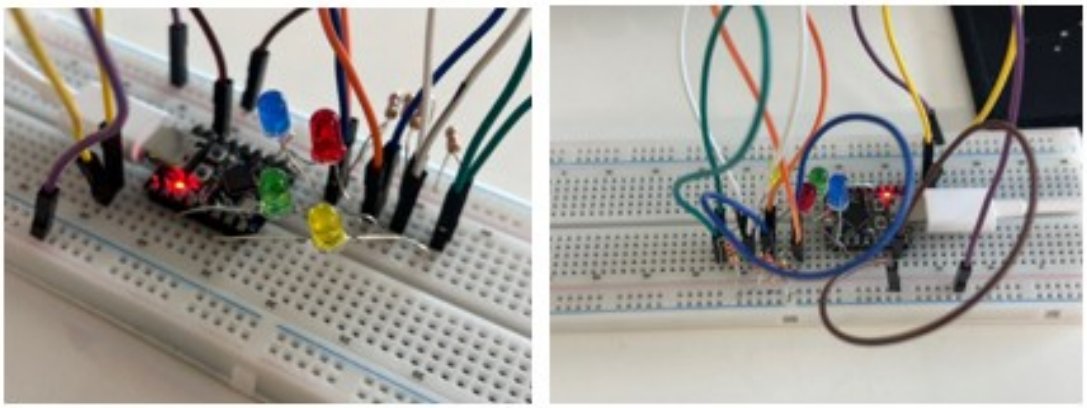
<표 3> 준비물 목록

배선 순서

① 브레드보드 중앙 홈에 보드를 걸치도록 꽂는다 — 양쪽 핀이 서로 다른 세로줄에 들어간다. ② LED 4 개를 각각 독립적으로 배선한다(점퍼선 공유 금지): LED 긴 다리(+)를 해당 GPIO 핀과 같은 세로줄에 직결하고, 짧은 다리(−)는 바로 옆 빈 세로줄에, 저항 한쪽은 그 세로줄에, 반대쪽은 각자 GND 레일까지 개별로 길게 연결한다. 저항 4 개를 한 점퍼선으로 묶어 GND 로 보내면 그 점퍼선 하나만 불량이어도 LED 4 개가 전부 안 켜지므로 반드시 개별 연결한다. ③ 부저(GND / VCC / S 3 핀)는 GND 를 GND 레일에, VCC 를 보드 3V3 핀에, S 를 GPIO4 핀에 각각 연결한다. ④ 보드 GND 핀을 점퍼선으로 브레드보드 GND 레일에 연결한다 — 이 선이 빠지면 전체 회로가 닫히지 않아 아무것도 켜지지 않는다. ⑤ 830point 브레드보드는 +/− 레일이 중앙에서 끊겨 있는 경우가 많으므로, 보드 GND 점퍼선과 각 저항의 GND 연결이 같은 쪽 레일에 있는지 반드시 확인한다.

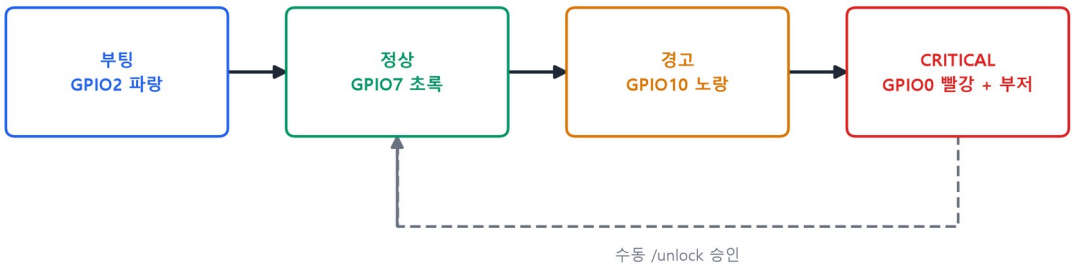
핀	용도	비고
GPIO2	파랑 — 부팅/Wi-Fi 연결 중	변경 없음
GPIO7	초록 — 정상	접촉 불량으로 GPIO3 에서 재배정
GPIO10	노랑 — 경고(1~2 단계 탐지)	GPIO1·8·9 도 후보였으나 GPIO8/9 는 스트래핑 핀이라 회피
GPIO0	빨강 — Lockdown(CRITICAL)	접촉 불량으로 GPIO6 에서 재배정
GPIO4	부저 — Lockdown 시에만 경보음	변경 없음

<표 4> ESP32-C3 핀 배치 (최종본)



<그림 3> 최종 배선 — 브레드보드 실측 사진(좌: 전체 배선, 우: LED/저항 근접)

핀을 최초 설계에서 재배정한 이유는 두 가지다. 첫째, 브레드보드 실측 과정에서 GPIO3·GPIO5·GPIO6 자리의 접촉 불량이 확인되어 새 칼럼으로 옮겨야 했다. 둘째, GPIO8·GPIO9 는 ESP32-C3 의 스트래핑 핀(부팅 모드 선택용)으로, 일반 I/O 로 사용할 경우 리셋 타이밍에 영향을 줄 수 있어 처음부터 배제하였다. 또한 각 LED 의 +(저항 이전, GPIO 직결) 쪽 배선을 서로 분리해야 한다는 점도 실측을 통해 확인하였다 — 같은 칼럼을 공유하면 두 LED 가 동시에 서로 다른 값을 출력할 때 GPIO 간 직접 충돌이 발생하여 깜빡이다 꺼지는 증상으로 나타났다.



<그림 4> 상태 LED 전이도 — 부팅→정상→경고→CRITICAL, /unlock 으로 수동 복귀

4-3-5. 펌웨어 구현 (MicroPython)

ESP32-C3 는 Wi-Fi 연결 완료 후 포트 80 에서 HTTP 서버로 동작하며, n8n 의 요청을 직접 받아 상태를 전환한다. 모든 POST 요청은 X-Auth-Token 헤더가 config.py 의 공유 토큰과 일치해야 처리되도록 하여, 외부에서 임의로 장치 상태를 조작할 수 없게 하였다.

Method	Path	인증	설명
GET	/	불필요	현재 상태 조회
POST	/device/control	필요	정상 운영 명령 통과(파랑 블링크). Lockdown 중에는 423 으로 거부
POST	/warning	필요	노랑 전환. 이미 Lockdown 상태면 무시(하향 방지)

Method	Path	인증	설명
POST	/lockdown	필요	빨강 고정 + 부저 경보
POST	/unlock	필요	수동 승인 시 초록으로 복귀

<표 5> ESP32-C3 HTTP API 엔드포인트

아래는 Lockdown 적용부의 핵심 로직이다.

```
def apply_lockdown():
    # 깜빡임 3회 + 부저 3번 울림 후 고정 빨강 + 경보음
    for _ in range(3):
        leds_off()
        buzzer.value(0)
        time.sleep(0.15)
        led_red.value(1)
        buzzer.value(1)
        time.sleep(0.15)
    apply_status("locked")
```

/device/control 로 들어온 정상 운영 명령은 파랑 블링크로 눈에 보이게 확인시키되, 이때 두 LED 가 동시에 켜지지 않도록 현재 상태 LED 를 잠깐 껐다가 복구하는 방식으로 GPIO 충돌을 구조적으로 방지하였다. 이는 4-3-4 에서 확인한 배선 단의 충돌 문제를 소프트웨어 레벨에서도 한 번 더 차단하기 위한 이중 안전장치다.

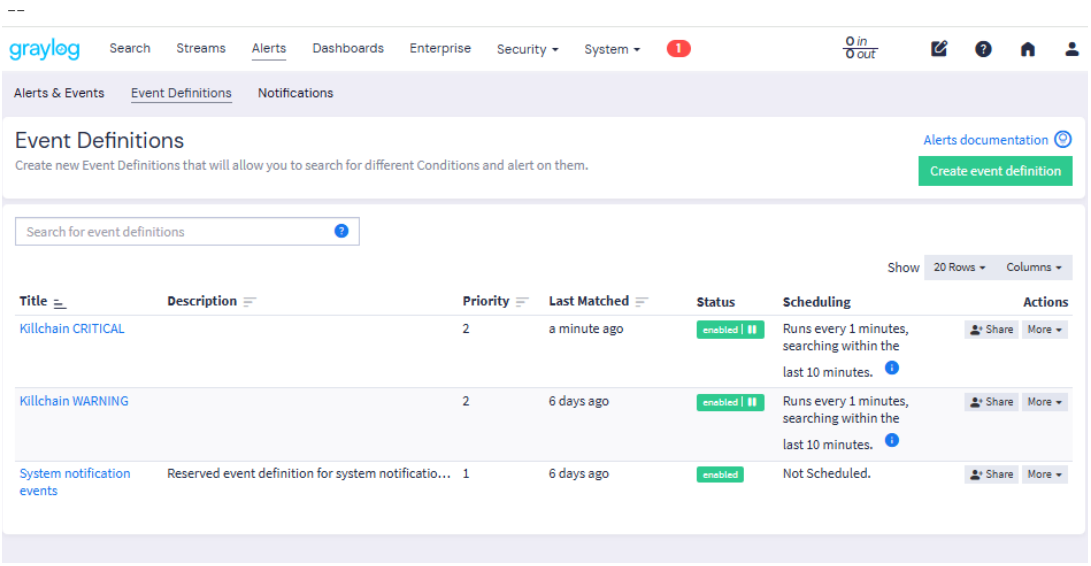
```
Invoke-RestMethod -Method Post -Uri "http://192.168.7.81/device/control" `
  -Headers @{ "X-Auth-Token" = "aleph4-killchain-2026" } `
  -Body '{"command":"test"}' -ContentType "application/json"
```

4-3-6. Graylog 상관분석 규칙과 시나리오 2 연동

Event Definition "Killchain CRITICAL"은 client_ip 를 기준으로 그룹핑하여, 서로 다른 event_type 이 10 분 윈도우 내 3 종 이상(card(event_type) >= 3) 모이면 발동한다(1 분 주기 실행, 5 분 Grace Period).

```
# 최초
event_type:(recon_scan OR api_key_compromised OR device_control)

# 시나리오 2 연동 후
event_type:(recon_scan OR PORT_SCAN OR api_key_compromised OR BRUTE_FORCE OR
device_control)
```



<그림 5> Graylog Event Definitions 목록 — Killchain CRITICAL / WARNING 등록 확인

시나리오 1(scenario2-middleware, 폴더명은 리네이밍 전 기준)의 PORT_SCAN/BRUTE_FORCE 실제 공격 이벤트가 시나리오 3 와 동일한 client_ip 필드를 사용한다는 점을 확인하고, 쿼리에 이벤트 타입만 추가하여 시나리오 1 의 실공격 데이터도 같은 킬체인 판정에 들어오도록 연동하였다. 다만 PORT_SCAN/BRUTE_FORCE 2 종만으로는 조건을 채우지 못하므로, 시나리오 2·3 단계 이벤트 중 최소 1 개가 같은 10 분 윈도우 안에 더해져야 CRITICAL 이 발동한다.

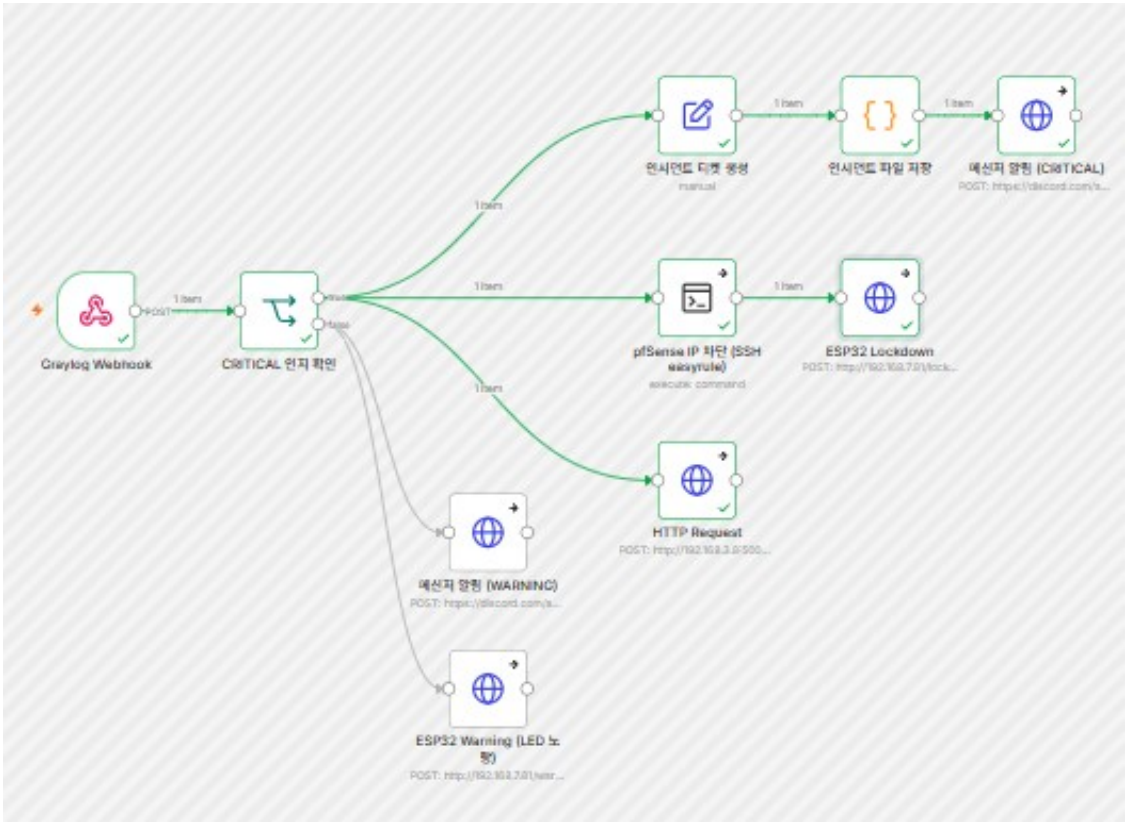
이 연동의 의의는 시나리오 3 가 더 이상 시나리오 3 자체의 합성 테스트 이벤트에만 반응하는 고립된 모듈이 아니라, 팀 전체 시나리오의 실제 공격 로그를 입력으로 받는 통합 상관분석 계층으로 확장되었다는 데 있다.

4-3-7. n8n 워크플로우 연동

"시나리오 3 - 킬체인 상관분석 대응" 워크플로우는 Graylog 알림을 웹훅으로 수신한 뒤, IF 노드("CRITICAL 인지 확인")로 분기한다. CRITICAL 분기에서는 ESP32 /lockdown 호출과 함께 공용 대시보드 티켓 생성, Discord 봇 CRITICAL 알림, pfSense 공격 IP 차단이 이어지고, 그 외 분기에서는 ESP32 /warning 호출과 Discord 봇 WARNING 알림만 수행한다.

ESP32 Lockdown/Warning 노드의 URL 과 X-Auth-Token 값을 실제 보드 IP(192.168.7.81)와 인증 토큰으로 교체하여, 플레이스홀더 상태였던 노드를 실동작 가능한 상태로 전환하였다. 이 작업 이전까지는 n8n 워크플로우가 논리적으로는 완성되어 있었으나 실제 하드웨어와는 연결되지 않은 상태였고, 이번 연동을 통해 Graylog 탐지부터 물리적 장치 반응까지의 전 구간이 처음으로 실동작하게 되었다.

연동 초기에는 pfSense 공격 IP 차단 노드가 SSH 인증 단계에서 반복적으로 실패하여 CRITICAL 분기의 나머지 노드(ESP32 Lockdown 포함)까지 함께 막히는 문제가 있었다. 이 문제의 원인과 해결 과정은 4-3-10 에서 별도로 다룬다. 아래 그림은 그 문제를 해결한 뒤, pfSense 차단·ESP32 Lockdown·인시던트 티켓 생성·Discord 봇 알림까지 CRITICAL 분기의 모든 노드가 성공으로 표시된 최종 실행 로그다.



<그림 6> n8n 워크플로우 최종 실행 로그 — CRITICAL 분기 전 노드(pfSense 차단·ESP32 Lockdown 포함) 성공

4-3-8. 동작 검증

보드가 Wi-Fi에 연결된 뒤, PowerShell에서 Invoke-RestMethod로 전체 상태 전이를 수동 검증하였다.

```
# 상태 조회
Invoke-RestMethod -Uri "http://192.168.7.81/"

# 경고 (노랑)
Invoke-RestMethod -Uri "http://192.168.7.81/warning" -Method Post `
  -Headers @{ "X-Auth-Token" = "aleph4-killchain-2026" }

# Lockdown (빨강 + 부저)
Invoke-RestMethod -Uri "http://192.168.7.81/lockdown" -Method Post `
  -Headers @{ "X-Auth-Token" = "aleph4-killchain-2026" }

# 수동 해제 (초록)
Invoke-RestMethod -Uri "http://192.168.7.81/unlock" -Method Post `
  -Headers @{ "X-Auth-Token" = "aleph4-killchain-2026" }
```

부팅(파랑) → Wi-Fi 연결 완료(초록) → /warning 호출 시 노랑 → /lockdown 호출 시 깜빡임 후 빨강 고정과 부저 → /unlock 호출 시 초록 복귀까지, 4가지 상태 전이를 모두 실물 LED로 확인하였다. 이 과정에서 Lockdown 상태에서는 /warning이 의도적으로 무시되도록 구현되어 있음을 재확인하였는데, 이는 이미 확정된 침입 대응 단계가 경고 단계로 임의 하향되지 않도록 하기 위한 설계다.

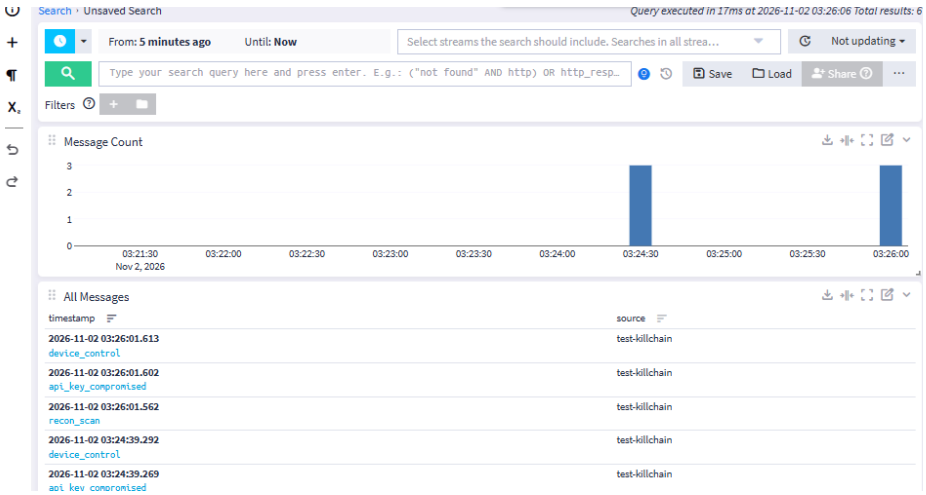
검증 항목	결과
GET / 상태 조회	정상 — device/status 필드 반환
POST /warning (정상 상태에서)	정상 — 노랑 전환 확인
POST /warning (Lockdown 상태에서)	정상 — 무시됨(하향 방지 로직 확인)
POST /lockdown	정상 — 3 회 깜빡임 후 빨강 고정 + 부저 경보
POST /unlock	정상 — 초록 복귀
POST /device/control (Lockdown 중)	정상 — 423 거부 확인

<표 6> ESP32-C3 API 수동 검증 결과

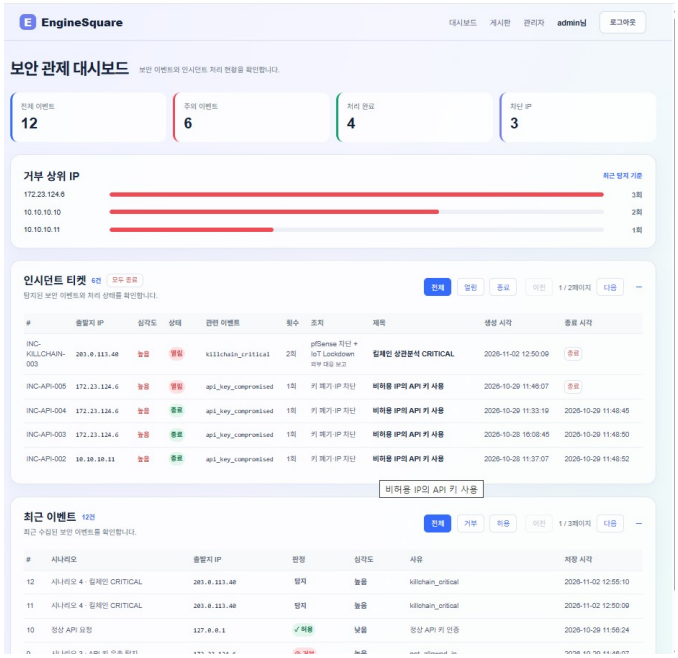
4-3-9. 자동 공격 재현 테스트

test_killchain.py critical 로 recon_scan → api_key_compromised → device_control 3 단계를 동일 공격 IP(203.0.113.188)로 순차 전송하여, Graylog 탐지부터 n8n 호출, 보드 자동 Lockdown 까지 사람 개입 없이 전 구간이 자동으로 동작함을 검증하였다.

```
cd scenario4-killchain/scripts
python test_killchain.py critical
```



<그림 7> Graylog 에 수신된 test_killchain.py 이벤트 (recon_scan / api_key_compromised / device_control)



<그림 8> 공용 대시보드에 생성된 킬체인 인시던트 티켓 (INC-KILLCHAIN-003)

전송 직후가 아니라 Graylog 의 1 분 실행 주기 내에 보드가 자동으로 빨강과 경보로 전환되는 것을 확인하였으며, 이는 "실제 공격이 발생하면 사람이 아무것도 누르지 않아도 하드웨어가 즉시 반응한다"는 자동 대응 체계를 입증하는 핵심 결과다.

> *참고: Graylog 의 10 분 검색 윈도우와 5 분 Grace Period 특성상, 수동으로 /unlock 한 뒤에도 같은 공격 IP 의 로그가 윈도우 내에 남아있으면 재탐지되어 다시 Lockdown 으로 전환될 수 있다. 이는 공격 증거가 남아있는 한 방어 태세가 유지된다는 의도된 동작으로, 시연 시 공격 주입 후 약 10 분 이내로 흐름을 구성하였다.*

오탐(false positive) 방지 로직도 함께 검증하였다. test_killchain.py warning 은 recon_scan 과 api_key_compromised 2 종만 전송하여 event_type 이 3 종에 도달하지 못하게 한다 — 이 경우 CRITICAL 로 격상되지 않고 WARNING(노랑)에 머물러야 정상이다. 실제로 이 명령을 실행했을 때 보드가 노랑에서 멈추고 빨강으로 전환되지 않는 것을 확인하여, 완전한 킬체인(3 단계)에 미달하는 공격 징후를 과도하게 CRITICAL 로 올리지 않는다는 점을 검증하였다.

```
python test_killchain.py warning
# recon_scan + api_key_compromised 2 종만 전송 -> WARNING(노랑) 유지, CRITICAL 로 격상되지 않음
```

4-3-10. 문제 발생 및 해결 과정

본 파트는 처음부터 완성된 설계로 진행된 것이 아니라, 하드웨어 배선·펌웨어·네트워크 각 층에서 반복적으로 문제를 확인하고 해결하는 과정을 거쳐 최종 형태에 도달하였다. 아래는 그 과정을 문제별로 정리한 것이다.

문제 1 빨강·초록·노랑 LED 가 전혀 켜지지 않음 (파랑만 정상)

원인 증상을 단계적으로 좁혀나갔다. ① 보드 전원 LED 는 켜져 있어 전원 공급 자체는 정상이었다. ② LED·저항을 여러 번 교체해도 동일 증상이 재현되어 부품 불량은 배제했다. ③ 브레드보드 없이 점퍼선과 부품만으로 3V3↔GND 직결 테스트를 해도 안 켜져 배선 로직 자체의 문제도 아니었다. ④ 펌웨어를 완전히 재설치(erase + reflash)해도 동일해 소프트웨어 문제도 배제했다. ⑤ 결정적으로 Thonny REPL

에서 b.value(1)/b.value(0)으로 파랑(GPIO2)을 직접 제어하자 코드대로 정상 동작했고, r.value(1)/g.value(1)/y.value(1)도 에러 없이 실행됐지만 실제 LED는 켜지지 않았다 — 즉 MCU가 핀에 신호를 내보내는 것 자체는 정상이었고, 문제는 그 3개 핀이 물려있던 브레드보드 칼럼의 물리적 접촉 불량이었다.

▷ **해결** 접촉이 확인된 새 칼럼(GPIO0·GPIO7·GPIO10)으로 LED·저항을 전부 재배선하고, config.py와 배선 가이드 문서를 동시에 갱신하였다.

참고: 위 재배선으로 해결되지 않는 경우를 대비해 다음 순서도 마련해 두었다. ④ LED 극성을 반대로 꽂아 긴 다리/짧은 다리 착각 가능성을 배제한다. ⑤ 3V3 대신 5V 핀으로 해당 LED만 직결해 보아, 켜지면 3V3 레귤레이터의 전류 한계를 의심하고 5V 기준으로 전환을 검토한다. ⑥ 멀티미터로 해당 칼럼의 GPIO 핀-GND 간 전압을 직접 측정해, 코드로 HIGH를 줬을 때 3V 근처가 나오지 않으면 핀 자체 손상으로 보고 다른 핀으로 재배정한다.

문제 2 빨강 LED가 파랑과 동시 점등 시 깜빡이다 꺼짐

원인 두 LED의 +(저항 이전, GPIO 직결) 쪽 배선이 같은 브레드보드 칼럼을 공유하여, 서로 다른 GPIO가 동시에 다른 전압을 출력할 때 핀 간 직접 충돌이 발생했다.

▷ **해결** 배선에서 anode 쪽 칼럼을 LED마다 완전히 분리하고, 펌웨어의 blink_once()도 두 LED가 동시에 HIGH 상태가 되지 않도록(상태 LED를 잠깐 끈 뒤 파랑만 깜빡이고 복구) 로직을 수정해 이중으로 방지하였다.

문제 3 GPIO8을 노랑 LED에 쓰려 했으나 부팅 안정성 우려

원인 GPIO8·GPIO9는 ESP32-C3의 스트래핑 핀(부팅 모드 선택용)으로, 일반 I/O로 사용 시 리셋 타이밍에 영향을 줄 수 있다.

▷ **해결** 스트래핑 핀이 아닌 GPIO10으로 최종 배정하였다.

문제 4 OSError: Wifi Internal State Error

원인 소프트 리셋 직후 MicroPython Wi-Fi 드라이버의 내부 상태가 비정상으로 남는 경우가 간헐적으로 발생했다.

▷ **해결** connect_wifi()에서 연결 시도 전 wlan.active(False) → 대기 → wlan.active(True)로 드라이버를 먼저 초기화하도록 수정하였다.

문제 5 /warning을 호출해도 LED 색이 바뀌지 않는 것처럼 보임

원인 오동작이 아니라 의도된 설계였다 — Lockdown 상태에서는 /warning 호출이 무시되도록 구현되어, 이미 확정된 대응 단계가 경고 단계로 임의 하향되지 않는다.

▷ **해결** /unlock으로 먼저 초기화한 뒤 /warning을 재호출하여 정상 동작(노랑 전환)임을 재확인하였다.

문제 6 n8n ESP32 노드가 플레이스홀더 URL/토큰 상태로 방치

원인 초기 워크플로우 설계 시점에는 실제 보드 IP가 확정되지 않아 <ESP32_IP>, <IOT_AUTH_TOKEN> 등 임시값으로 남아 있었다.

▷ **해결** 보드의 실제 IP(192.168.7.81)와 AUTH_TOKEN으로 노드 값을 교체하고 워크플로우를 재게시하였다.

문제 7 n8n → pfSense SSH 연동이 반복적으로 handshake 타임아웃

원인 pfSense SSH credential의 Host/Port/계정 정보는 모두 정상이었고, n8n의 자체 "Test connection"도 통과했다. 그런데 실제 노드 실행(Execute step)에서는 매번 "Timed out while waiting for handshake"로 실패했다. n8n 컨테이너 안에서 시스템 OpenSSH 클라이언트로 동일한 계정·명령(easyrule block wan ...)을 직접 실행하면 즉시 성공했으므로, 네트워크·pfSense·인증정보 문제는 모두 배제되었다. 남은 차이는 접속에 사용하는 클라이언트뿐이었다 — n8n 이 내장한 SSH 노드(ssh2, Node.js 라이브러리, Password 인증)가 pfSense sshd와 암호화 알고리즘 협상(key exchange)에 실패해 멈추는 것으로 결론지었다.

> ▷ **해결** n8n 컨테이너 안에서 Ed25519 키 쌍을 생성해 pfSense admin 계정의 Authorized SSH Keys에 등록하고, n8n credential 타입을 Password에서 Private Key로 교체하였다. 이후 test_killchain.py critical로 전체 파이프라인을 재현하자 pfSense 차단·ESP32 Lockdown을 포함한 CRITICAL 분기 전 노드가 한 번에 성공하는 것을 확인하였다(그림 6).

이 과정에서 확인한 공통적인 교훈은, 겉으로 같은 증상(LED 미점등, 네트워크 타임아웃)이라도 원인이 소프트웨어 로직·배선·핀 선정·특정 라이브러리의 호환성 중 어디에 있는지에 따라 해결 방법이 전혀 다르다는 점이다. 매번 가능한 원인을 하나씩 배제해 범위를 좁히는 isolation 절차(REPL로 핀 직접 제어, 시스템 ssh 클라이언트로 동일 명령 재현 등)가 디버깅 시간을 실질적으로 단축하는 데 유효했다.

4-3-11. 시연 절차

실제 발표에서는 수동 명령어로 각 상태를 하나씩 소개한 뒤, 마지막에 자동 공격 재현으로 마무리하는 순서를 사용한다. Thonny는 보드를 USB 시리얼로 직접 제어할 때만 필요한 도구이며, main.py는 전원이 들어오는 즉시 자동 실행되므로 발표 PC에는 설치하지 않는다 — 오히려 Thonny가 연결된 상태에서 실수로 중단시키면 보드의 HTTP 서버 자체가 꺼지는 사고로 이어질 수 있음을 실제로 확인하였다.

순서	내용
①	부팅 — 파랑. RST 버튼으로 재부팅(명령어 없음). 전원 인가 직후 자동으로 파랑, Wi-Fi 연결 시도 중을 의미
②	정상 — 초록. Wi-Fi 연결 성공 시 자동 전환(명령어 없음). 평상시 대기 상태
③	상태 조회 — GET /. 지금 보드가 어떤 상태로 인식하고 있는지 JSON으로 확인
④	경고 → Lockdown → 해제를 수동 호출로 하나씩 시연하며 각 색의 의미를 설명
⑤	디바이스 제어 호출 — 파랑 1회 깜빡임으로 평상시 명령 수신 확인 동작을 소개
⑥	실제 공격 재현 — ① 경고용 IP로 정찰·인증 공격 2종을 전송해 노랑(WARNING) 확인 후 수동 해제, ② 새 IP로 장치 제어까지 3종을 전송해 CRITICAL 확인. pfSense 차단 표가 0건에서 1건으로 늘고, ESP32 빨강 고정·Discord 봇 알림·대시보드 티켓이 자동으로 이어지는 전체 흐름을 증명

<표 7> 시연 데모 시퀀스

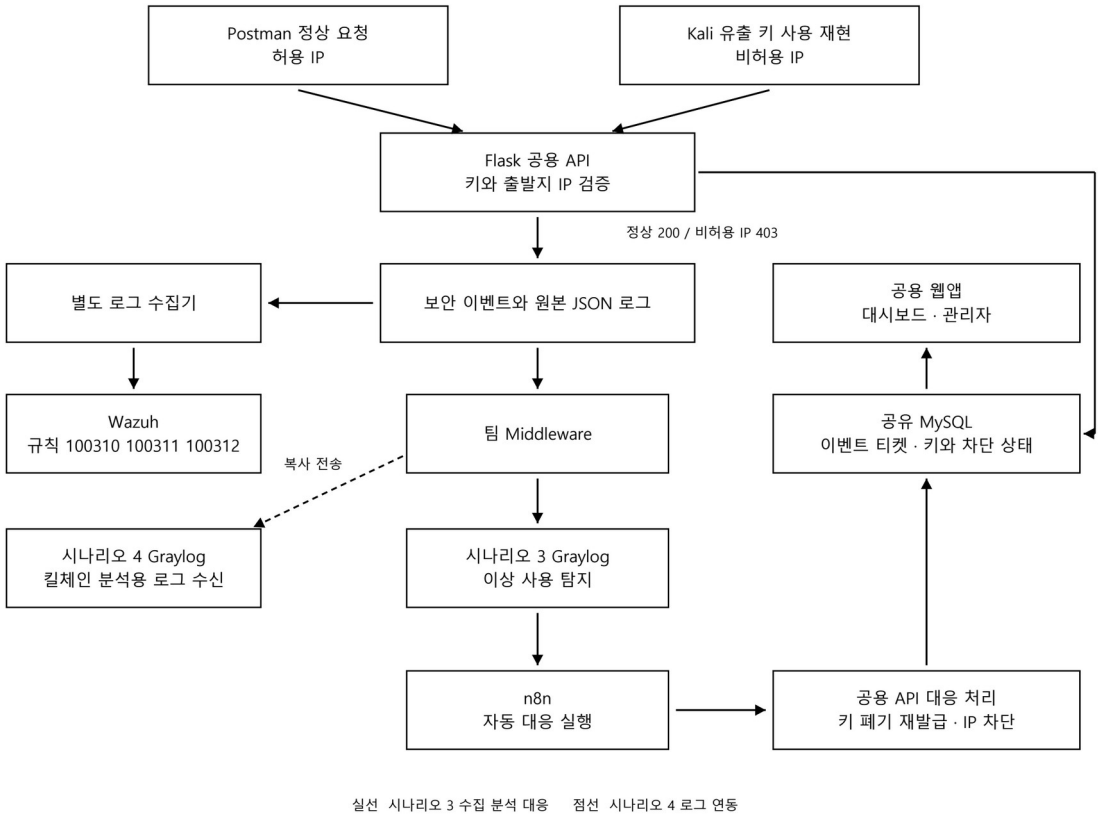
4-3-12. 한계 및 향후 발전 가능성

현재 규칙은 서로 다른 유형이 3 종 모였는지만 판정하고 순서는 보지 않으며, Lockdown 상태는 보드 메모리에만 있어 재부팅 시 초기화되고, n8n 노드에 보드 IP 가 직접 들어 있다. 개선 방향은 순서까지 판정하는 규칙, 시나리오 1·2 의 실제 공격을 동일 IP·동일 윈도우로 맞춘 통합 재현, pfSense 이중화와 다중 장비 확장이다.

더 나아가 본 구조는 관제실 벽에 시계처럼 걸린 LED 패널로 여러 구역의 위험도를 한눈에 보여주거나, 출입문과 같은 물리 장치에 대한 API 공격을 감지했을 때 해당 장치를 잠그고 상태를 LED 로 표시하는 방향으로 확장할 수 있다. 이는 구현을 마친 내용이 아니라 향후 구상이다.

5. 통합 아키텍처 (+ 공용 대시보드 연동 스펙)

세 시나리오는 독립적으로 구현하되, 로그 수집(시나리오 1) → 탐지·분석(Graylog, 시나리오 2·3) → 자동 대응(n8n) → 공용 기록(MySQL·웹앱) 흐름으로 통합하였다. 아래 구성도는 시나리오 2(유출 API 키)를 중심으로 전체 데이터 흐름을 보여주며, 그림 내 "시나리오 3/4 Graylog" 표기는 번호 변경 전 자료이므로 본 보고서 기준 시나리오 2(이상 사용 탐지용)·시나리오 3(킬체인 상관분석용) Graylog 로 읽는다.



<그림 5-1> 통합 아키텍처 — API 요청부터 자동 대응·공용 화면까지

5-1. 로그 수집·전달 계층

API 서버와 IoT 로그는 팀 Middleware(시나리오 1)를 거쳐 Graylog 로 전달된다. Middleware 는 GRAYLOG_GELF_URLS 에 복수 목적지를 설정할 수 있어, 각 시나리오 담당자의 로컬 Graylog(이상 사용 탐지용)와 상관분석 담당자의 Graylog(킬체인 분석용)에 동일 이벤트를 동시에 전달한다 — 주 전송

경로의 지연을 피하기 위해 보조 목적지는 비동기 백그라운드로 복제 전송한다. Wazuh 는 원본 JSON 로그를 별도 경로로 수집·분석하며 현재 구성에서는 n8n 을 직접 호출하지 않는다.

5-2. 탐지·상관분석 계층

Graylog Event Definition 은 시나리오별로 분리되어 있다. 시나리오 2 용 규칙은 api_key_compromised·HTTP 403 이벤트를 기준으로 탐지하며, 시나리오 3 용 규칙(Killchain CRITICAL/WARNING)은 client_ip 기준 그룹핑과 event_type 3 종 이상(10 분 윈도우)을 기준으로 탐지한다. 두 규칙 모두 전용 n8n Webhook 을 호출하는 구조로 분리되어, 한 시나리오의 탐지 로직 변경이 다른 시나리오에 영향을 주지 않는다.

5-3. 자동 대응 오케스트레이션(n8n)

시나리오	n8n 호출 대상	수행 액션
시나리오 2	공용 API /automation/revoke-key	기존 키 폐기, 교체 키 발급, 공격 IP 차단
시나리오 3 CRITICAL	pfSense(SSH) · ESP32(HTTP) · 공용 API · Discord 봇	WAN 차단, Lockdown(LED+부저), 티켓 생성, 알림
시나리오 3 WARNING	ESP32(HTTP) · Discord 봇	Warning(LED 노랑), 알림

<표 5-1> 시나리오별 n8n 대응 액션

5-4. 공용 대시보드 연동 스펙

각 시나리오의 n8n 워크플로우는 공용 API 에 결과를 POST 하여 하나의 대시보드·DB 에서 조회되도록 한다. 인시던트 번호는 시나리오별 접두어로 구분하되(INC-API-nnn, INC-KILLCHAIN-nnn), 공통 필드(event_type, client_ip, severity, status, timestamp)는 동일한 스키마를 따르도록 맞추었다.

항목	스펙
인시던트 번호	시나리오 2: INC-API-nnn / 시나리오 3: INC-KILLCHAIN-nnn
공통 필드	event_type, client_ip, severity(LOW/WARNING/CRITICAL), status, timestamp
중복 처리	동일 event_type·client_ip·(키 식별자) 요청은 기존 티켓에 연결하고 횟수를 누적
상태 구분	자동 대응 완료와 티켓 종료를 분리 — 티켓 종료가 차단 해제·키 복원을 의미하지 않음

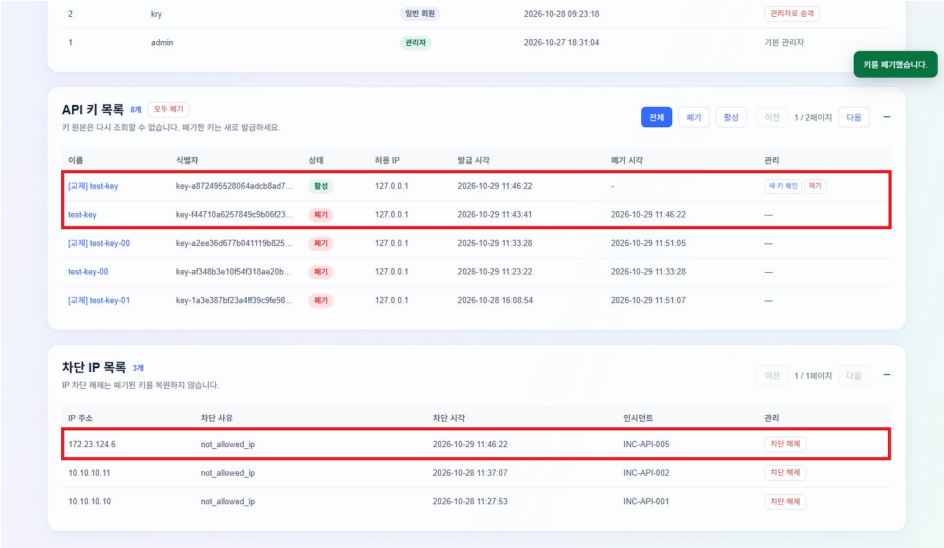
<표 5-2> 공용 대시보드 연동 공통 스펙

> *참고: 시나리오 3 의 n8n → 공용 대시보드 연동(HTTP Request 노드, /automation/killchain-response)은 11/2~11/3 작업에서 실제 호출을 확인하였다. incident_id 포맷·timestamp 필드가 서버 검증 규칙과 정확히 일치하는지는 리허설 전 공용 API 담당(김라연)과 함께 한 번 더 교차 점검이 필요하다.*

6. 실제 구현 화면

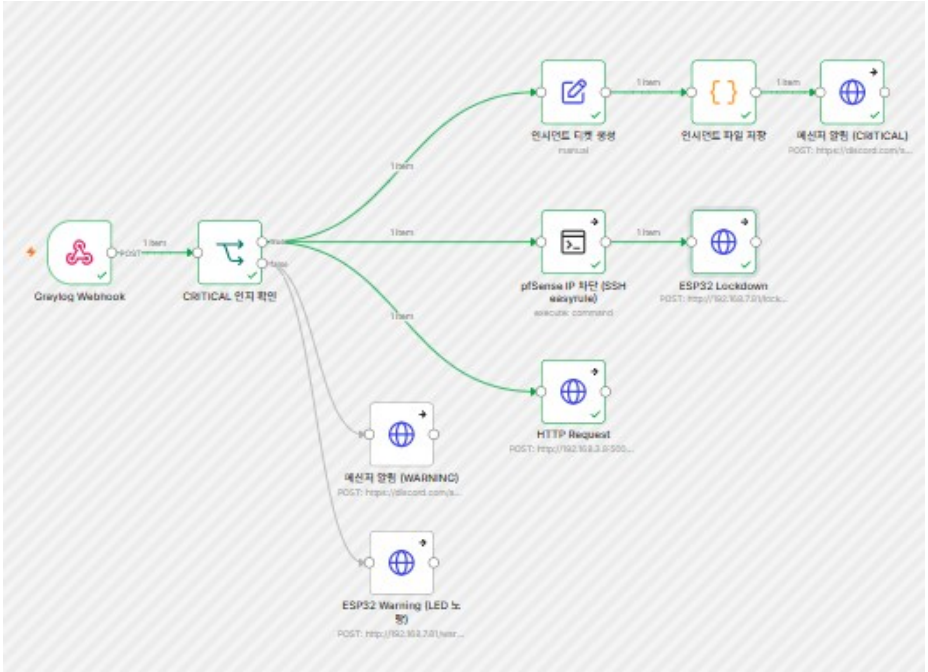
시나리오별 핵심 화면을 모았다. 세부 캡처·설명은 각 4-N 절에 포함되어 있으며, 여기서는 통합 관점에서 대표 화면만 정리한다.

6-1. 시나리오 2 — 유출 API 키 탐지/대응

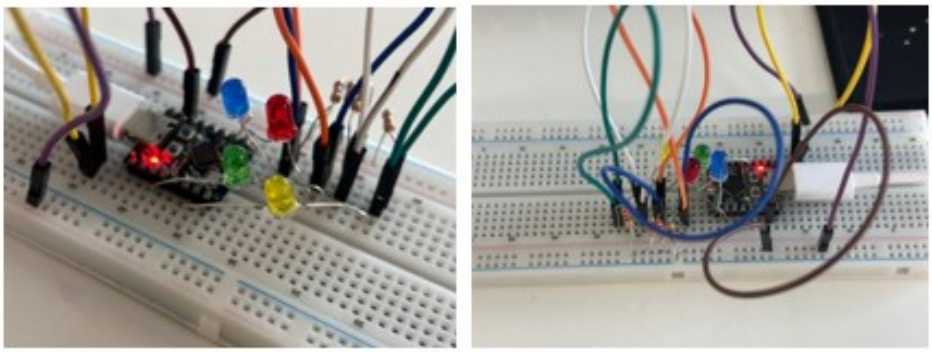


<그림 6-1> 관리자 화면 — 키 폐기·교체 상태와 INC-API-005 IP 차단 기록

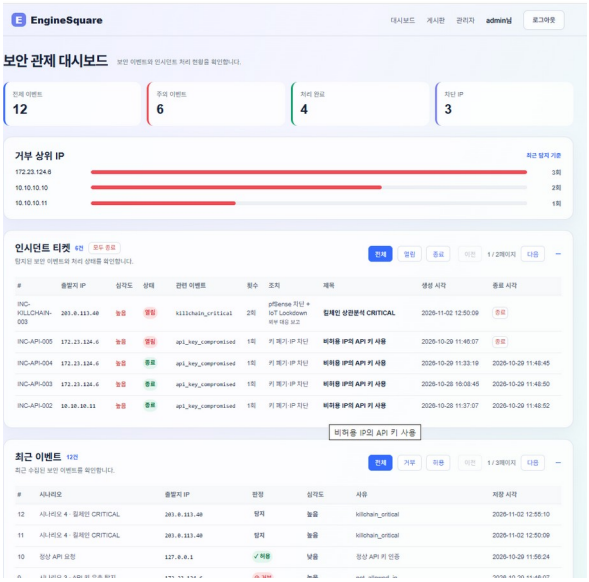
6-2. 시나리오 3 — 킬체인 상관분석/하드웨어



<그림 6-2> n8n 워크플로우 실행 — CRITICAL 분기 전 노드 성공(pfSense 차단·ESP32 Lockdown 포함)



<그림 6-3> ESP32-C3 상태 LED 실물 배선



<그림 6-4> 공용 대시보드 킬체인 인시던트 티켓(INC-KILLCHAIN-003)

참고: 시나리오 1(미들웨어)은 콘솔/로그 중심 구현이라 화면 캡처보다는 4-1 절의 수치(표 4-1-1, 4-1-2)로 결과를 제시한다. 공용 웹앱의 메인/로그인/게시판/대시보드/관리자 화면 캡처는 담당자(김라연)가 추가하면 된다.

7. 통합 테스트 결과

개별 시나리오 검증(4-1~4-3)과 별개로, 여러 시나리오가 같은 파이프라인·공용 자원을 함께 쓸 때의 동작을 통합 관점에서 확인하였다.

7-1. 다중 Graylog 전송 검증

시나리오 1 Middleware 의 다중 목적지 전송(GRAYLOG_GELF_URLS) 기능을 적용한 뒤, 동일 공격 로그가 각 담당자의 로컬 Graylog 와 상관분석용 Graylog 양쪽에 모두 도착하는지 확인하였다. 기존 단일 전송 경로를 유지한 채 상관분석용 전송을 추가하는 방식이라, 기존 시나리오 2 탐지 파이프라인에 영향을 주지 않고 시나리오 3 킬체인 분석에 필요한 로그를 확보할 수 있음을 검증하였다.

검증 항목	조건	결과
다중 전송 도달	동일 이벤트를 2 개 Graylog 로 동시	양쪽 모두 수신 확인

검증 항목	조건	결과
	전송	
대량 부하 안정성	20,000 건 연속 인입(시나리오 1 부하 테스트)	유실 0 건, 전송오류 0 건
상관분석 조건 확장	시나리오 1 PORT_SCAN/BRUTE_FORCE 를 킬체인 쿼리에 포함	event_type 3 종 조건에 정상 반영

<표 7-1> 다중 Graylog 전송 통합 검증

7-2. 동일 공격자 기준 전체 킬체인 통합 재현

시나리오 3 의 자동 공격 재현 스크립트(test_killchain.py)로 동일 client_ip 에 대해 recon_scan → api_key_compromised → device_control 3 단계를 순차 발생시켜, Graylog 탐지부터 n8n 호출, pfSense 차단, ESP32 Lockdown 까지 전 구간이 자동으로 동작함을 확인하였다(4-3-9, 4-3-10 참고). 시연에서는 시나리오 1 의 시뮬레이터(spike_test_demo.py)가 만든 PORT_SCAN·BRUTE_FORCE 이벤트에 test_killchain.py device 의 장치 제어 이벤트를 더하는 방식으로, 경고용 IP 와 위급용 IP 를 나누어 WARNING(노랑)과 CRITICAL(빨강)을 각각 확인하였다. 이 테스트들은 시나리오 3 쪽에서 발생시킨 이벤트로 수행한 것이며, 시나리오 2 의 실제 유출 키 사용까지 동일 IP·동일 10 분 윈도우로 맞춰 처음부터 끝까지 재현하는 전체 통합 재현은 추가로 진행할 계획이다.

7-3. 공용 API·대시보드 상태 공유 검증

시나리오 2 에서는 폐기 키 재사용 거부(401), 교체 키 정상 요청(200), 관리자 화면의 상태 변경이 서버 재시작 없이 즉시 반영됨을 확인하였다. 시나리오 3 의 킬체인 결과는 공용 API 를 통해 티켓·이벤트 화면에 표시되도록 연동하였으며, 장비(ESP32·pfSense) 대응은 시나리오 3 전용 n8n 에서 처리하도록 역할을 구분하였다.

7-4. 남은 통합 검증 항목

항목	담당
동일 공격자 IP 로 시나리오 1·2·3 전 구간 실제 재현(합성 이벤트 아닌 실공격 트래픽 기준)	전원
incident_id·timestamp 포맷이 공용 API 검증 규칙과 정확히 일치하는지 재확인	유지원 · 김라연
공용 웹앱 외부 PC 접속(방화벽/바인딩) 재확인	김라연
pfSense SSH 키 인증 상태 리허설 당일 재확인(환경 리셋 가능성)	유지원

<표 7-2> 리허설 전 남은 통합 검증 항목

8. 겪은 문제 및 해결 과정

개별 기능을 공용 환경에 연결하는 과정에서 로그 전송 안정성, 상태 공유 및 장비 연동 문제가 발생하였다. 시나리오 1 은 Middleware, 시나리오 2 는 유출 API 키 탐지·자동 대응, 시나리오 3 은 킬체인·IoT 를 기준으로 주요 문제와 해결 과정을 정리하였다. 시나리오 3 의 세부 문제-원인-해결 7 건은 4-3-10 절에 별도로 정리되어 있다.

8-1. 대량 로그 처리의 연결 안정성

시나리오 1 에서는 2 만 건 연속 부하 시뮬레이션 중 포트·소켓 고갈 문제가 발생하였다. 짧은 시간에 반복되는 연결과 세션 관리가 전송 안정성에 영향을 주었으며, 수집 경로의 브로커 지연 조회 방식도 개선이 필요하였다. 클라이언트 연결 풀링과 세션 정리를 보완하고 브로커 lag 캐시를 비동기로 처리하도록 개선하여, 2 만 건 연속 인입 시 전송오류 0 건을 확인하였다.

8-2. 분리된 Graylog 환경과 로그 연동

담당자마다 로컬 Graylog 를 사용하여 킬체인 상관분석에 필요한 단계별 로그가 한곳에 모이지 않는 문제가 있었다. 기존 로그 전달을 유지하면서 상관분석 담당자의 Graylog 에도 같은 이벤트를 전달할 필요가 있었다. Middleware 에서 복수 GELF 목적지 설정을 지원하고 보조 Graylog 에는 비동기 백그라운드 복제 전송을 적용해, 주 전송 경로의 지연 없이 양쪽 Graylog 에 도착함을 확인하였다. 외부 PC 접속 문제는 웹앱의 바인딩 주소와 Windows 방화벽 설정을 점검하여 조정하였고, Graylog 알림에서 공격 IP 가 비어 표시되는 문제는 event.group_by_fields 를 참조하도록 수정하였다.

8-3. 공용 API 와 관리 화면의 상태 공유

시나리오 2 와 공용 웹앱에서는 키 폐기·교체, IP 차단·해제 및 티켓 상태를 같은 기준으로 관리해야 했다. 기존 SQLite 에서 공유 MySQL 로 이전하고 API 와 웹앱이 같은 키·차단 정보를 사용하도록 구성하여, 서버 재시작 없이 변경이 반영되도록 하였다.

8-4. Wazuh 실시간 로그 전달 누락

API·웹앱을 실행한 뒤에도 Wazuh 에서 새 공격 이벤트가 조회되지 않았다. 확인 결과 API 로그를 Wazuh 에 전달하는 별도 수집기가 통합 실행 파일에 포함되지 않아 실행되지 않은 상태였다. 통합 실행 파일에 수집기의 시작·종료와 전용 로그 저장을 추가하고, API 로그 파일이 없으면 생성될 때까지 기다리도록 보완하였다. 이후 신규 Kali 요청에서 규칙 100310(레벨 10)과 INC-API-011 을 확인하여 실시간 수집·탐지를 검증하였다.

8-5. IoT LED 배선·pfSense 연동 장애와 최종 해결

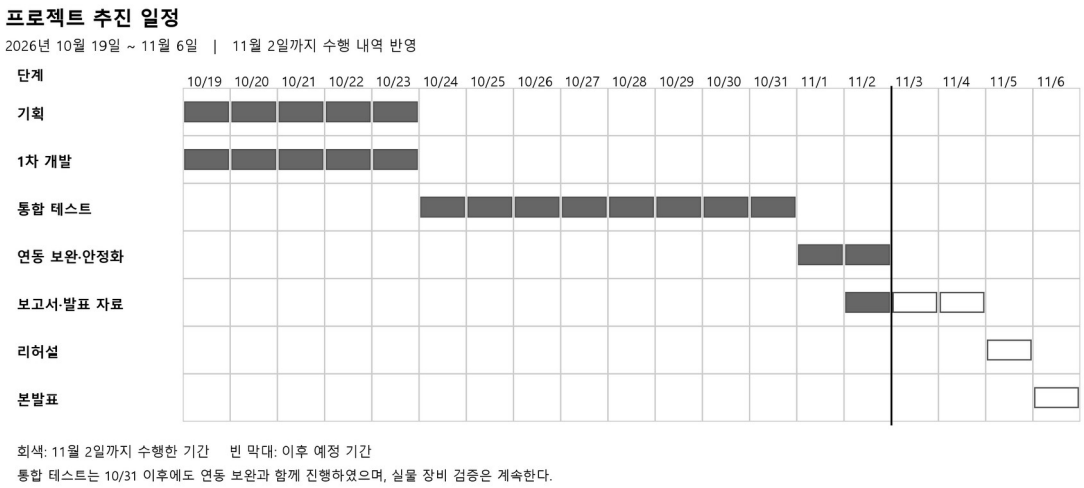
시나리오 3 의 ESP32-C3 LED 일부가 정상 동작하지 않아 부품·저항·점퍼선과 극성을 교차 점검하였다. 브레드보드·핀 접촉 불량을 원인으로 특정하고, GPIO3/5/6 대신 GPIO0/7/10 을 사용하도록 배선과 설정을 변경하였다.

11 월 2 일 시점에는 ESP32 배선 안정화까지만 완료되고, n8n→pfSense 공격 IP 차단 연동은 SSH 핸드셰이크 타임아웃으로 미해결 상태였다. 11 월 3 일 추가 디버깅으로, n8n 내장 SSH(ssh2) 라이브러리와 pfSense sshd 간 암호화 알고리즘 협상 실패가 원인임을 특정하고 SSH Password 인증을 Private Key(Ed25519) 인증으로 교체하여 해결하였다. 이후 test_killchain.py critical 로 Graylog 탐지→n8n→pfSense 차단→ESP32 Lockdown 전 구간 자동 성공을 확인하여, 시나리오 3 의 실물 장비 통합 대응이 최종 완료되었다(4-3-10 문제 7 참고).

9. 진행 일정 및 역할 분담

시나리오별 개발과 공용 환경 통합을 병행하였다. 11 월 2 일까지의 주요 수행 내역과 리허설·본발표 준비 일정을 정리하였다.

9-1. 단계별 추진 일정



<그림 9-1> 프로젝트 추진 일정(간트 차트)

9-2. 주요 수행 경과

기간	핵심 수행 내용
10/19~23	시나리오별 담당 역할·검증 계획을 확정하고 기획서를 제출. API→Middleware→Graylog→n8n 1 차 연동 확인
10/26~27	공통 이벤트 형식과 Graylog 적재·검색 점검. 로그인·게시판·티켓·최근 이벤트 화면 구성, 시나리오별 결과를 하나의 대시보드로 통합 결정
10/28	회원가입·관리자·필터·페이지 이동 추가, 공유 MySQL 로 이전. 티켓 종료·차단 해제·키 상태 변경이 서버 재시작 없이 반영되도록 구성
10/29~30	시나리오 2 Wazuh 규칙과 공격·대응 증적 정리. Middleware 전송 안정성 보완, 시나리오 3 pfSense 연동과 LED 배선 점검
11/2	킬체인 결과 수신용 공용 API·티켓·이벤트 연동. 인시던트 번호·동시 실행 환경 정리, 외부 PC 접속과 양쪽 Graylog 전송 확인
11/3	시나리오 3 n8n→pfSense SSH 연동 최종 해결(키 인증 전환), 전체 파이프라인 자동

기간	핵심 수행 내용
	동작 확인. 허니토큰(구 시나리오 1) 제외 확정, 시나리오 3 개 체제로 재정리. 보고서 통합

<표 9-1> 주요 수행 경과

9-3. 담당 역할 및 수행 업무

담당	주요 역할	핵심 수행 업무
강아름	시나리오 1 Middleware	로그 수집·버퍼링·재전송 및 연결 안정성 개선, 다중 Graylog 전송 및 2 만 건 부하 테스트
김라연	시나리오 2 인증·인가 / 공용 웹앱·API·DB	유출 API 키 탐지 및 키 교체·IP 차단 자동화, 공용 웹앱·API·DB 구축 및 시나리오 통합
유지원 (팀장)	시나리오 3 킬체인·IoT	킬체인 상관분석 및 pfSense·ESP32 대응 연동, 하드웨어 안정화 및 통합 시연·발표 자료 정리, 회의록 작성

<표 9-2> 담당 역할 및 수행 업무

9-4. 후속 일정

리허설 전 시연 PC 와 계정·API 키·명령어를 준비하고, 각 시나리오의 초기 상태와 로그 수신 경로를 점검한다. API 공격 시연을 반복할 때에는 관리자 화면에서 차단 IP 와 키 상태를 점검하고 필요한 초기화를 수행한다. 전체 킬체인은 동일 공격자 IP 로 단계별 로그를 발생시켜 상관분석과 대응 결과를 확인하며, 시연 실패에 대비한 캡처·녹화 자료도 함께 준비한다.

일정	준비 및 확인 사항
11/3~4	보고서·발표 자료·시연 명령어 정리, 캡처·녹화 증적 준비
11/5 까지	시연 PC·계정·초기 상태 점검, 성능 튜닝 및 부하 결과 정리. 하드웨어 안정화와 킬체인 통합 재현, 보고서·발표 자료 최종 정리
11/5 리허설	리허설
11/6 본발표	본발표 — 개인별 담당 파트 설명 및 결과물 제출

<표 9-3> 후속 일정

10. 결론 및 향후 계획

10-1. 결론

본 프로젝트는 로그 수집의 안정성(시나리오 1), 인증·인가 이상 사용 탐지(시나리오 2), 다단계 공격의 상관분석과 물리적 장비 대응(시나리오 3)을 각각 구현하고, 공용 웹앱·API·DB 로 통합하여 하나의 보안 관제 흐름으로 연결하였다. 2 만 건 연속 부하에서 유실 0 건을 달성한 로그 버퍼링, 유출 API 키의 실시간 탐지와 자동 키 교체·IP 차단, 그리고 Graylog→n8n→pfSense→ESP32 로 이어지는 완전 자동화된 킬체인 대응까지 — 세 시나리오 모두 실제 동작을 증적(로그, 스크린샷, 실행 결과)으로 확인하였다.

특히 개발 과정에서 발생한 문제들(소켓 고갈, 분산된 Graylog, Wazuh 수집기 누락, LED 배선 접촉 불량, SSH 라이브러리 호환성 등)은 추측이 아니라 단계적으로 원인을 배제해 나가는 isolation 절차로 해결하였다는 공통점이 있다. 이는 8 장에 정리된 문제-원인-해결 기록과 4-3-10 절의 상세 디버깅 과정에 반영되어 있다.

10-2. 향후 계획

항목	내용
단기 (리허설 전)	7-4 절 남은 통합 검증 항목(동일 IP 전 구간 재현, incident_id 포맷 재확인, pfSense 키 인증 상태 재확인) 완료
중기	시나리오 1 브로커의 다중 서버 수평 확장(Kafka/Redpanda 전환), 시나리오 2 허용 IP 내 키 악용 탐지 기준 추가
장기	실제 공격 패턴 다양화(허니토큰 등 추가 탐지 시나리오 재도입 검토), pfSense 이중화 및 ESP32 다중 장비 확장, 관제실 벽 LED 패널·출입문 등 물리 장치 연동(향후 구상)

<표 10-1> 향후 계획

세 시나리오 모두 "탐지되면 자동으로 대응하고, 그 결과가 사람이 확인할 수 있는 형태로 남는다"는 동일한 설계 원칙을 따른다. 향후 시나리오를 추가하더라도 이 원칙(공용 인시던트 스펙, n8n 오케스트레이션, 공용 대시보드 기록)을 그대로 재사용할 수 있다는 점이 본 프로젝트 구조의 확장성이다.